REN-210: Managing the Rhapsody Environment

Users and Security

Overview

The best practice is to create individual user accounts for all the people who will access Rhapsody. This provides a number of benefits:

- It permits the creation of user accounts with limited access to Rhapsody. For example, a help desk user might only log into the Management Console to view limited functions.
- It makes it possible to see who has edited or is currently editing a Rhapsody component.
- It allows a user to tailor the way in which they receive notifications for the components within Rhapsody they have been subscribed to.
- It supports the use of Lockers to restrict user access to a part of the Rhapsody configuration.

This lesson will describe the process of creating and managing users within the system and creating user groups to allow you to tailor access rights. The lesson will also cover how lockers can be used to partition access to various parts of a Rhapsody configuration.

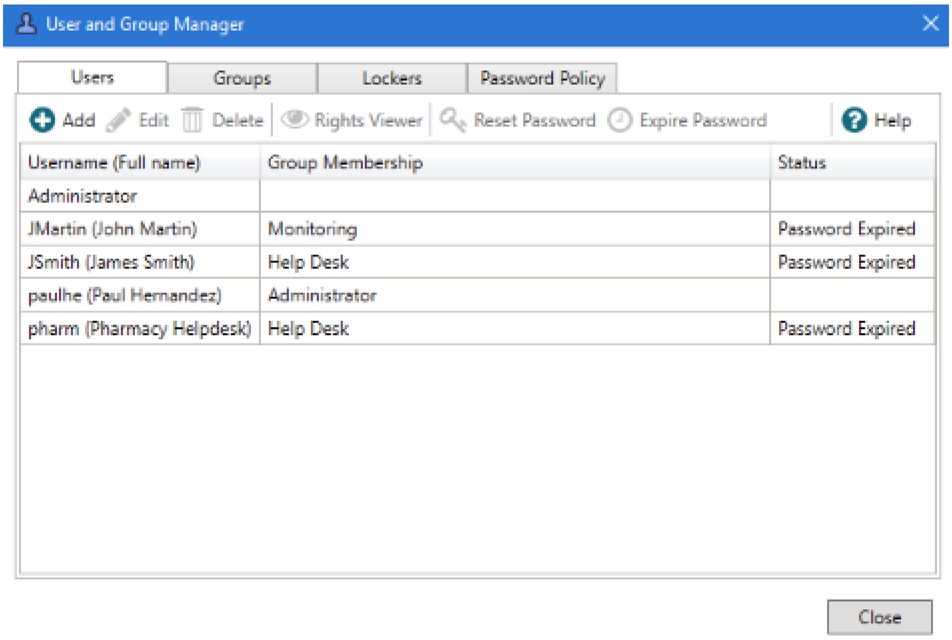
These functions are all accessed from the **User and Group Manager** screen. This is accessed from the **View** menu by selecting the **User Manager** option.

Lockers

A Rhapsody **Locker** allows an administrator to separate a solution's routes, communication points, and definition files from all other components on the engine. This option helps ensure that a solution's components are easily identified and reduces the risk that they will be accidentally changed while updating another project. When security is applied to a locker, users lacking appropriate access cannot modify the configuration of its contents, nor view the messages they process.

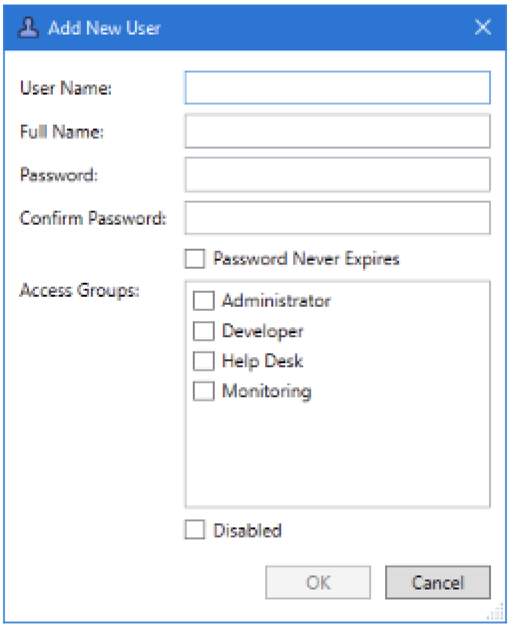
Users

Users and **Groups** are managed from the Rhapsody IDE via the **User Manager** screen, accessible from the **Rhapsody** menu item.



Adding a new user

A new user is added by selecting the **User** tab and selecting **Add**. This presents the **Add New User** dialog that is used to enter the details for the new user.



A new user needs to be provided with a **User Name** and **Password** so that they can log in to Rhapsody. The **Full Name** field should be populated to assist matching the user name with a real person or system. The **Password Never Expires** checkbox indicates that any password expiry rules associated with the password policy for this system should not apply to this account. This is most commonly used when the account is used by an automated system to access the Rhapsody API's.

For the account to be useful, it must be associated with one or more **Access Groups**. An access group defines a set permission within Rhapsody.

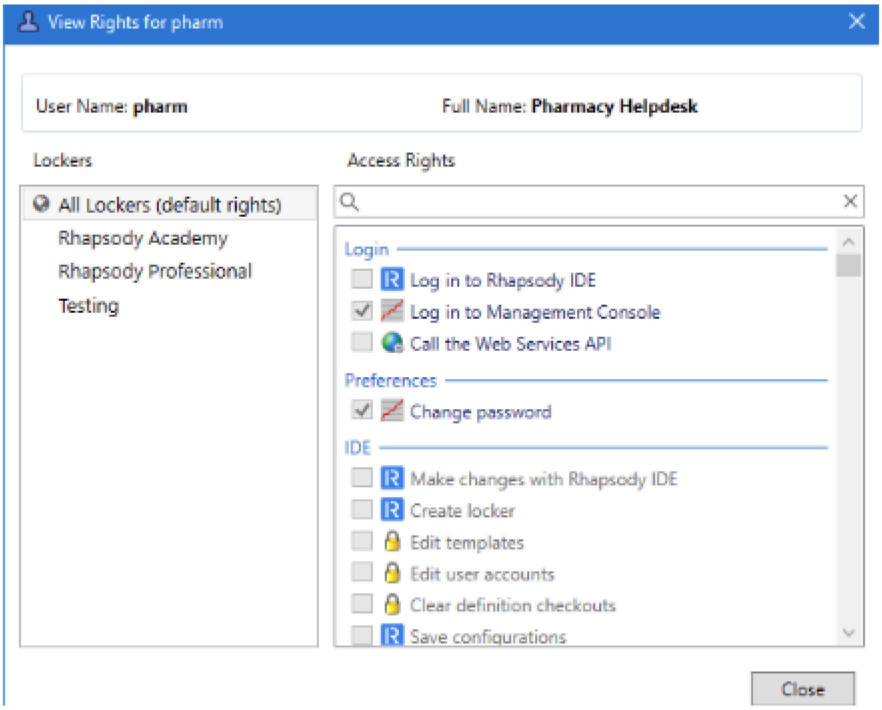
The **Disabled** checkbox is used to disable an account without the need to otherwise edit or delete it. This may be useful in a number of situations, including when a user is temporarily suspended or when suspicious activity is detected against an account.

Editing an existing user account

An existing user account is edited by selecting it from the list and selecting **Edit**. This presents the **Edit User** dialog box identical to the new user dialog, with the exception that the username can not be changed. All options behave the same way on this screen.

Viewing a user's rights

Selecting the **Rights Viewer** allows an administrator to view the permissions applied to an account. This can be used to confirm that a user's account has been correctly configured with the required access permissions.

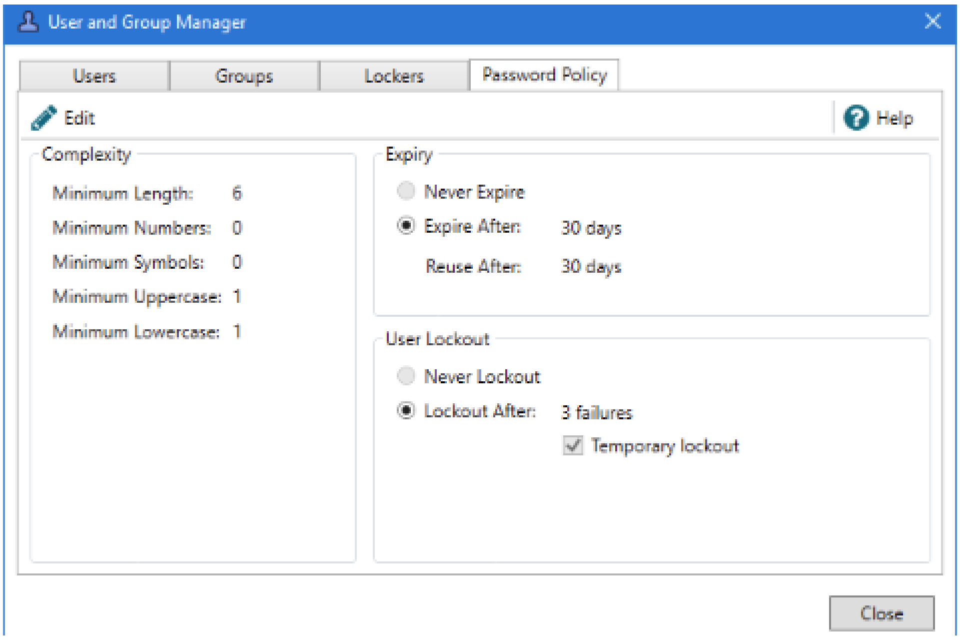


Permissions for an individual locker can be viewed by selecting the locker name from the list in the left panel.

Managing passwords

Reset Password allows an administrator to reset the password for a user, with an option to force the user to change the password when they next log in. **Expire Password** forces a user to change their existing password when they next log in.

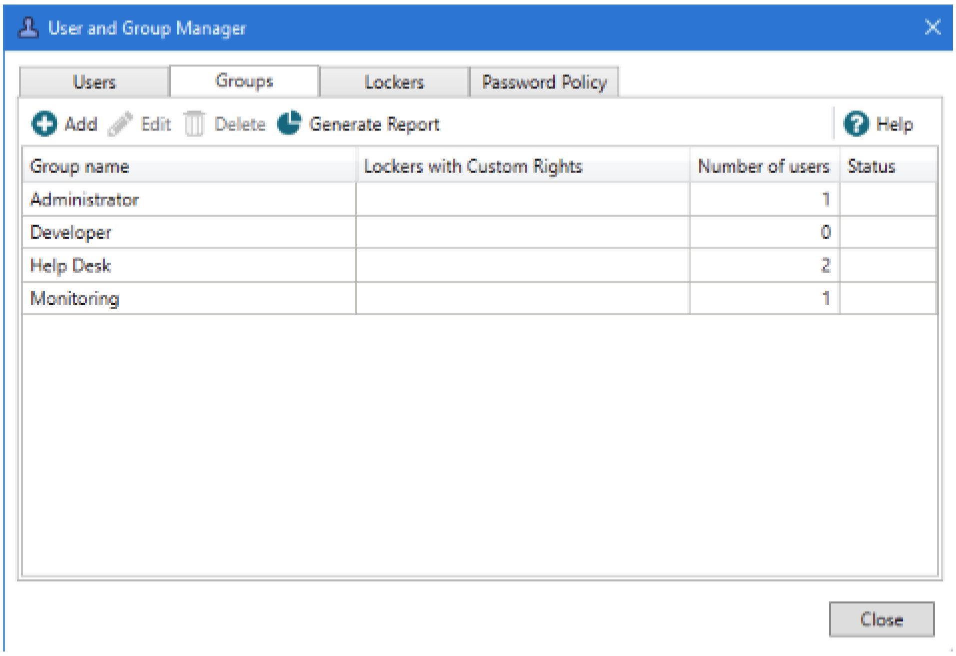
The **Password Policy** tab is used to view and edit the password policy that should apply to all accounts on the system.



Edit will open a dialog where the policy settings can be edited. If a password policy is changed to make it more restrictive, users who have passwords not compliant with the new policy will be forced to change them at the next login.

Groups

User groups allow a set of users to be assigned the same set of permissions within Rhapsody. By using groups, different users can easily be assigned the same level of access within Rhapsody. The **Groups** tab is used to add and edit groups.



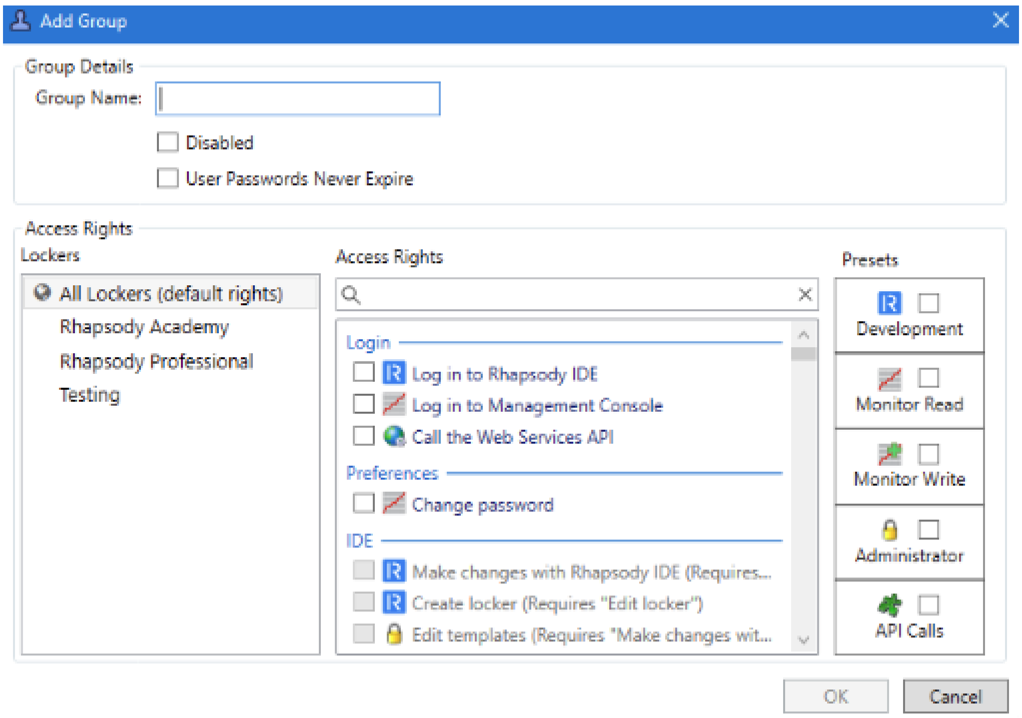
Default Groups

By default, four groups exist in Rhapsody:

- **Administrator** - the default permissions for this group enable a user to perform administrative tasks on the engine, such as managing users and access rights, managing backup schedules, managing licenses (it also includes all the monitoring and developer group permissions).
- **Dashboard** - the default permissions for this group provide access to the information that a Rhapsody Dashboard instance needs in order to monitor Rhapsody engines.
- **Developer** - the default permissions for this group enable a user to log into the Rhapsody IDE, view and manage configurations, and perform tasks that allow them to create routes and interfaces for the engine (it also includes the monitoring permissions).
- **Monitoring** - the default permissions for this group enable a user to log into the Management Console, view and manage messages, and perform tasks that allow them to monitor the day-to-day health of the engine.

Adding a new group

A new group is added by selecting the **Group** tab and selecting **Add**. This presents the **Add Group** dialog that is used to enter the details for the new user.



A new group should be provided with a meaningful name via the **Group Name** field. The **Password Never Expires** checkbox indicates that any password expiry rules associated with the password policy for this system should not apply to accounts associated with this group. The **Disabled** checkbox is used to disable a group without the need to otherwise edit or delete it.

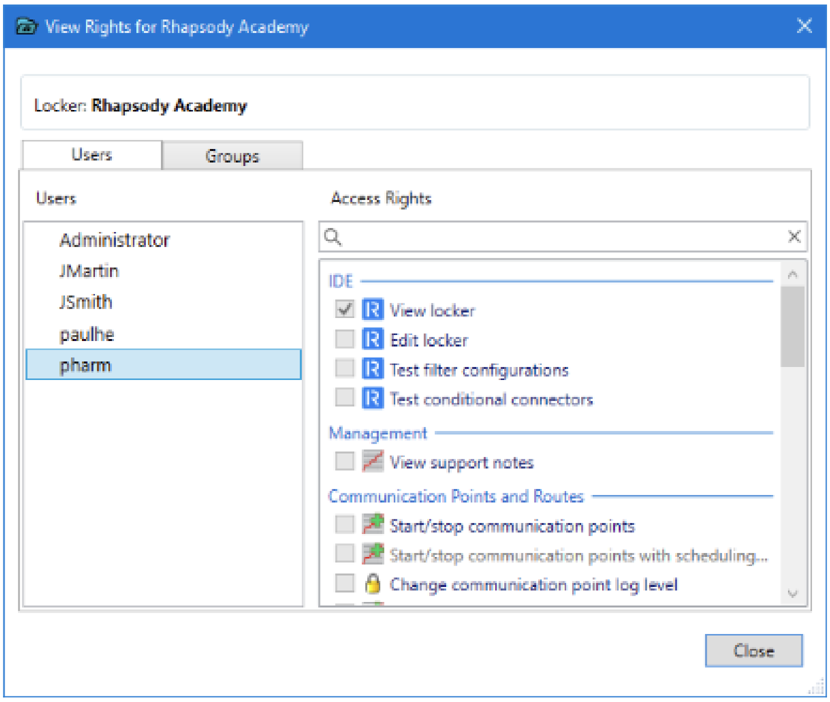
Access Rights

The **Access Rights** section of the Groups screen is used to define the rights that a group member will have within Rhapsody. This section of the screen is broken into three panels:

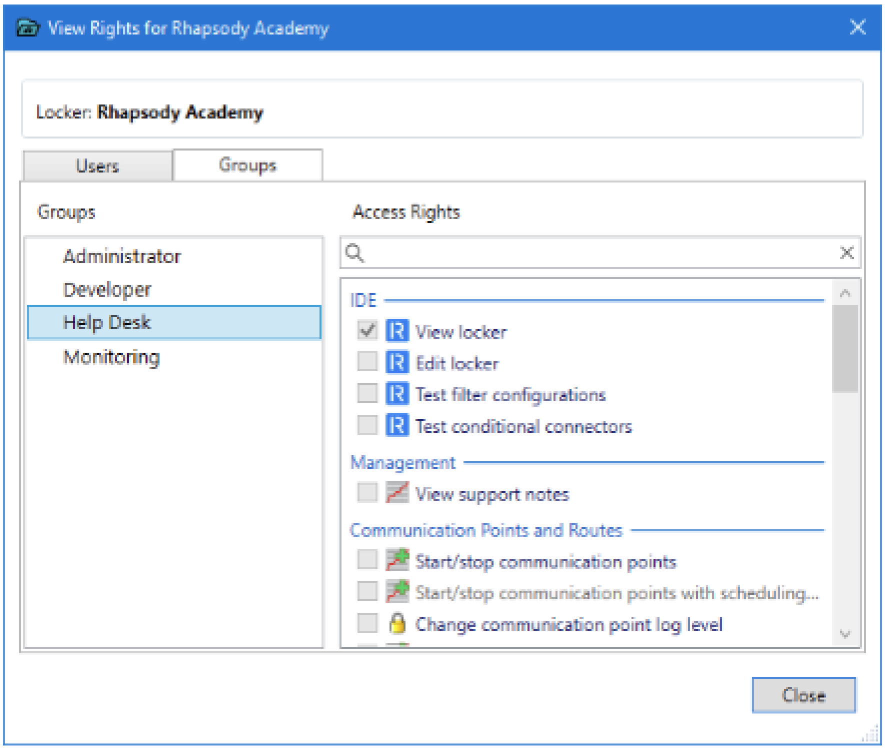
- The **Lockers** panel determines the locker that the access rights will be applied. By default, access rules are applied to the entire engine. Where permissions are applied to a specific locker, an icon is added in the panel alongside the name to indicate that there are access rights specific to this locker.

- The **Access Rights** panel lists the individual permissions for the group. The full set of access rights along with uses can be found in the documentation.

- The **Presets** panel lists a set of default access settings for various common roles. Selecting a preset will select all the access rights appropriate for the named role. The actual permissions granted by this group can then be edited from the **Access Rights** panel as required.
- **Editing an existing group**
- An existing group is edited by selecting it from the list and selecting **Edit**. This presents an **Edit Group** dialog box which is identical to the new group dialog. All options behave the same way on this screen.
- The right-click menu allows Groups to be copied and pasted. This is useful when you need a new group with a similar setup to an existing group without much of the effort required to set up a new group from scratch.
- **Lockers**
- Lockers allow you to compartmentalize your configuration. For example, you can use lockers to segregate views of patient data from users, thereby restricting access to patient data to authorized individuals. They provide various levels of access to sets of components and interfaces, so you can monitor the status of all interfaces while being able to manage a sub-set of alternative interfaces that you are responsible for maintaining.
- **View lockers rights**
- From the **Lockers** tab it is possible to view the rights a user has to a particular locker by selecting the desired locker, then selecting **Rights Viewer**. Rights can be viewed for a user, who may be a member of multiple groups, or an individual group.



User Rights



Group Rights

Managing Version Control

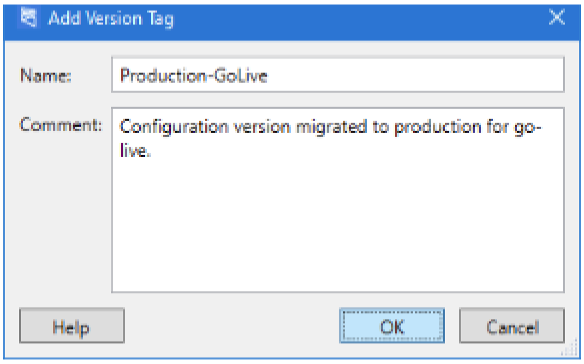
Making Version Control Effective

Version control is powerful method for tracking the changes that have occurred in an environment. To get the best out of version control the following practices should be followed.

Tag Milestones

Tags can be used to mark milestones or other significant changes in the **Rhapsody** configuration, such as the version of the configuration used in a production go-live or major upgrade, or that completes implementation and testing of a major new feature.

Any previously checked-in version within **Rhapsody** can be assigned a tag from the **Configuration History** screen. Select the version to tag and select **Add Tag**.



Make Changes Traceable

The name of the user who has performed each check-in operation is recorded as part of the check-in. To make this information useful, every **Rhapsody** user who might update configuration in an environment should have a unique user account.

Make Commits Understandable

Every check-in version should include a comment that describes the changes that have been made. A well written comment summarizes the check-in to help you and others understand the reason for the change. It is common for comments to include references to an external document or issue number that further describes the reason for the change.

Rhapsody includes a number of options that can be used to enforce rules around comments, which will be described later in this lesson.

Check in Frequently

Each check-in should relate to no more than one logical change to the Rhapsody configuration. The check-in should occur at the point when the components have been created or altered and tested. The act of checking in also saves all changes to the

Rhapsody server, protecting the changes from a system crash or network error that might cause the **Rhapsody IDE** to quit.

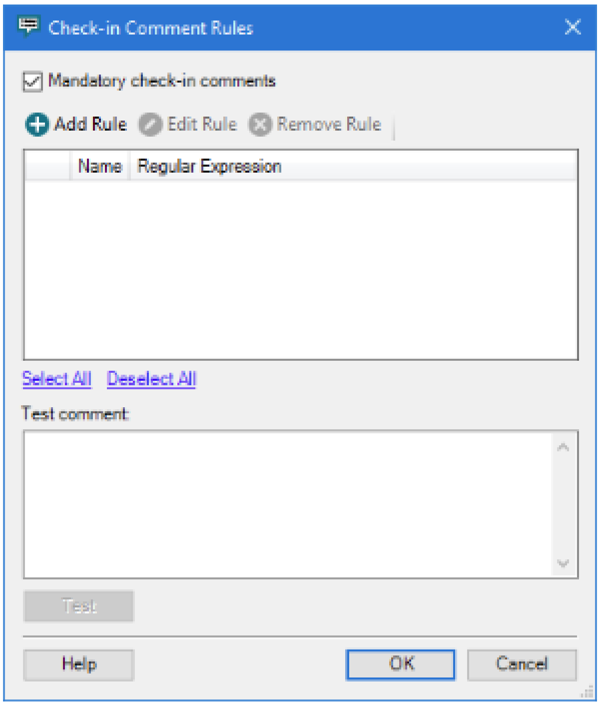
Check-in Comments

It can be useful to mandate check-in comments in an environment where it is important that every configuration change is documented (for example, a production environment). By default, a check-in comment is optional.

Check-In Comments are managed by opening the **Rhapsody** menu in the IDE and selecting the **Check-in Comment Rules** option.

Setting Comments as Required

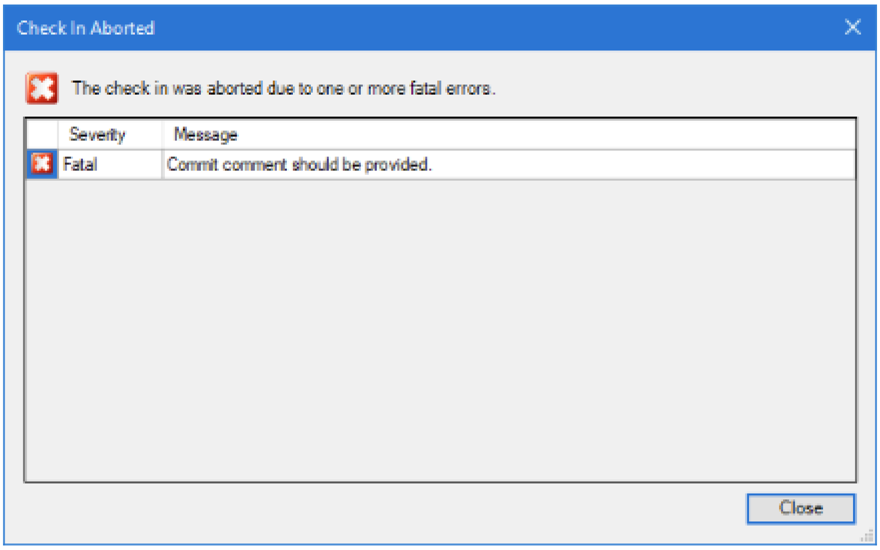
Comments can be made a requirement by opening the **Check-in Comment Rules** dialog box and selecting the **Mandatory check-in comments** checkbox.



Check-in Comment Rules dialog with Mandatory check-in comments selected

Attempting to Check In Without a Comment

An aborted check in dialog is displayed if an attempt is made to perform a comment that does not match the check-in comment rules. The dialog box includes information that details the reason for the failure.



The Aborted Check In error message

Comment Rules

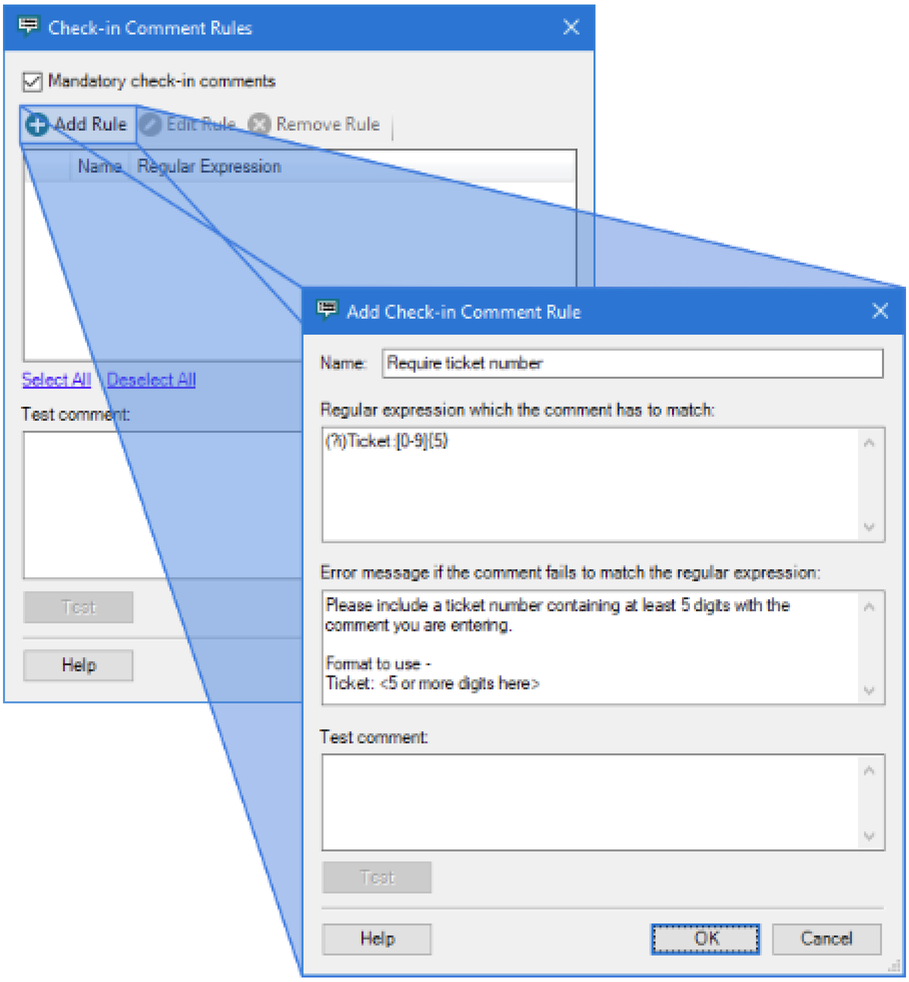
Comment Rules are used to control the content of the check-in comment. Comment rules are managed by opening them from the **Check-in Comment Rules** dialog. Multiple rules can be created where there is a requirement that check-in comments contain multiple items (for example, ticket number and reviewer).

Check-in comment rules are defined per environment. This lets you specify different rules for your development, test, and production Rhapsody Engine. For example, in production you might create a rule that checks to make sure a change control number has been entered. This can help prevent accidental check-ins to the wrong environment (as a check-in intended for a different environment would not be expected to contain the required detail).

Adding a Comment Rule

A new rule can be created even if the **Mandatory check-in comments** checkbox is not selected, but the rule itself will not come into force until this option is selected.

1. A new rule is created by selecting the Add Rule button. This opens a dialog where the rule is defined and tested.



Add Check-in Comment Rule dialog

A rule consists of a **Regular Expression** that the check in comment has to match. Consult the Rhapsody Administration Manual for a number of examples you can use, or visit the Java Regular Expression Library (opens in a new tab)for a summary of the constructs that are available as you create your own.

In the example above, the regular expression of (?i)Ticket:[0-9]{5} specifies that the comment must include the string 'Ticket:' followed by at least 5 digits; otherwise it will fail validation. The comment is not case-sensitive, does not have to be on its own line, and can include additional characters.

The **Error message** is displayed to the user during check-in if the comment rule fails. This should be populated with information that helps a user correct their comment.

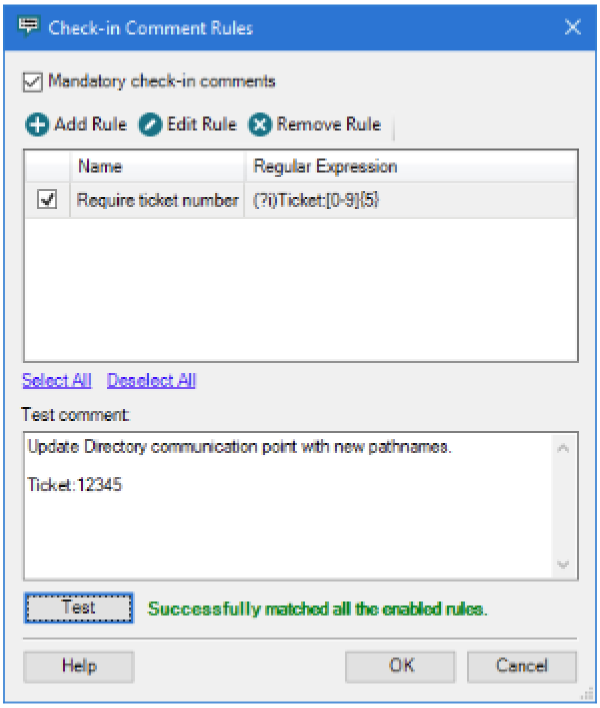
The **Test comment** panel can be used to test the regular expression before saving the rule.

2. Selecting the **OK** button saves the rule and closes the dialog. As the rule is saved, the regular expression is validated. An error message will be displayed if it does not parse correctly.

Testing the Rule

On the **Check-in Comment Rules** dialog, ensure that the rule(s) being tested are enabled by selecting their corresponding checkboxes. Enter a sample check-in comment, then select **Test**.

A message will be returned identifying whether the comment matched the conditions of the rule. If a failure occurs, the rule or rules that caused it will be highlighted with a red background.



A successful test of the check-in comment rule

Testing the Rule at Check-in

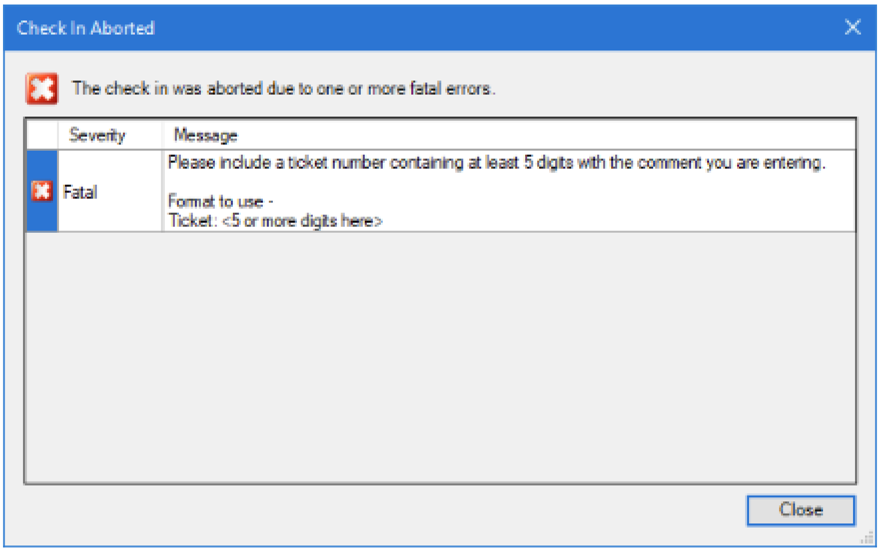
When a user enters a comment during check-in that does not match the specified regular expression, a Fatal Error results. The resulting screenshot includes the error message entered by the administrator at the time the comment rule was created.

If a user entered the following comment at check in:

Added No-operation filter to the route layout.

ticket:A12345

The following error message would be displayed.



An check-in that has been aborted due to a comment rule

Exercise - Check-in rules

Introduction

In this exercise, we will create two Check-in comment rules. The rules will ensure that every check-in contains an appropriate ticket number and reviewer name.

Below is an example of a comment that would satisfy the rules we will create:

Updated mapper definition to split with over-length OBX.5 fields into repeating fields.

Ticket: CSM-97

Reviewer: Erine

Create Ticket Rule

We will start by creating a rule to check that a ticket number has been supplied with the check-in comment. From the Check-in Comment Rules dialog, we select Add Rule.

Regular Expression Rule

For the ticket rule, we want the comments to contain the following pattern:

Ticket: xxx-xxxx

Within our Regular expression, we will check for the word Ticket: (making it case-insensitive for the benefit of people entering comments) followed by an optional space then one or more word or dash (-) characters.

Error Message

A comment should be added to the error message panel that explains why the check-in has not succeeded.

Test Rule

To make sure your regular expression is working as expected, make sure you test a range of comments, some of which you would expect to match and some should not match.

Here are some test comments to get you started:

- A ticket number has not been included:

Updated mapper definition to split with over-length OBX.5 fields into repeating fields.

Reviewer: Erine

- The ticket text is included but not a number:

Updated mapper definition to split with over-length OBX.5 fields into repeating fields.

Ticket:

Reviewer: Erine

- The ticket text and number have been included:

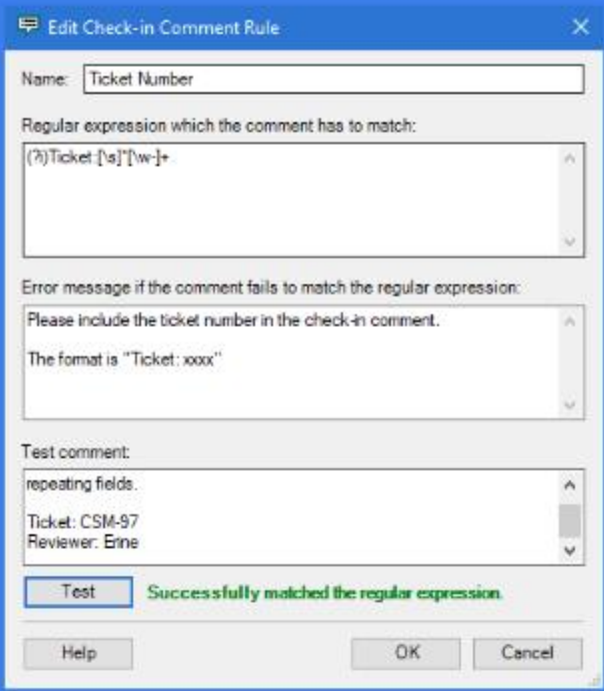
Updated mapper definition to split with over-length OBX.5 fields into repeating fields.

Ticket: CSM-97

Reviewer: Erine

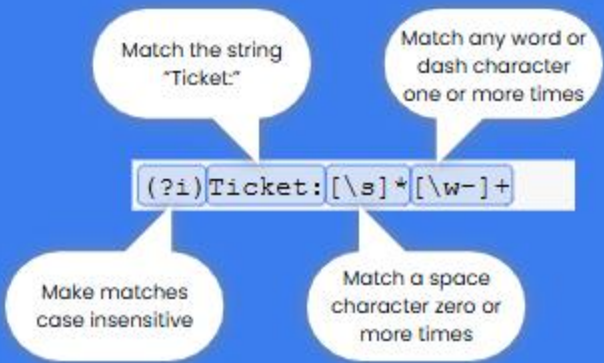
Model Answer

The completed rule should look similar to this:



Completed Ticket Check-in Rule

An explanation for the regular expression follows:



Regular Expression Explanation

Regular expression fragment	Description
(?i)	Make matches case-insensitive
Ticket:	Match the string "Ticket:"
[\s]*	Match a space character zero or more times *
[\w-]+	Match any word \w or dash - character one or more times +

When you test the rule, you might notice that any text you enter after Ticket: is valid. It would be possible to modify the regular expression to ensure that the ticket number matches a particular pattern. We haven't added that additional validation in this exercise to keep the regular expression simple.

Create Reviewer Rule

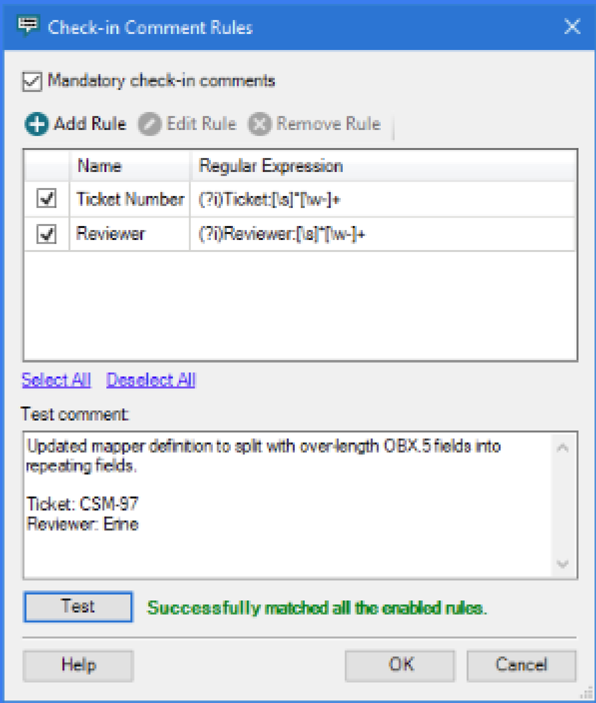
We now need to create a second rule that checks to make sure appropriate reviewer details have been added to the check-in comment. The pattern for valid reviewer details in the comment are:

Reviewer: name

As with the ticket rule, make sure you test your regular expression against a range of both valid and invalid test comments.

Model Answer

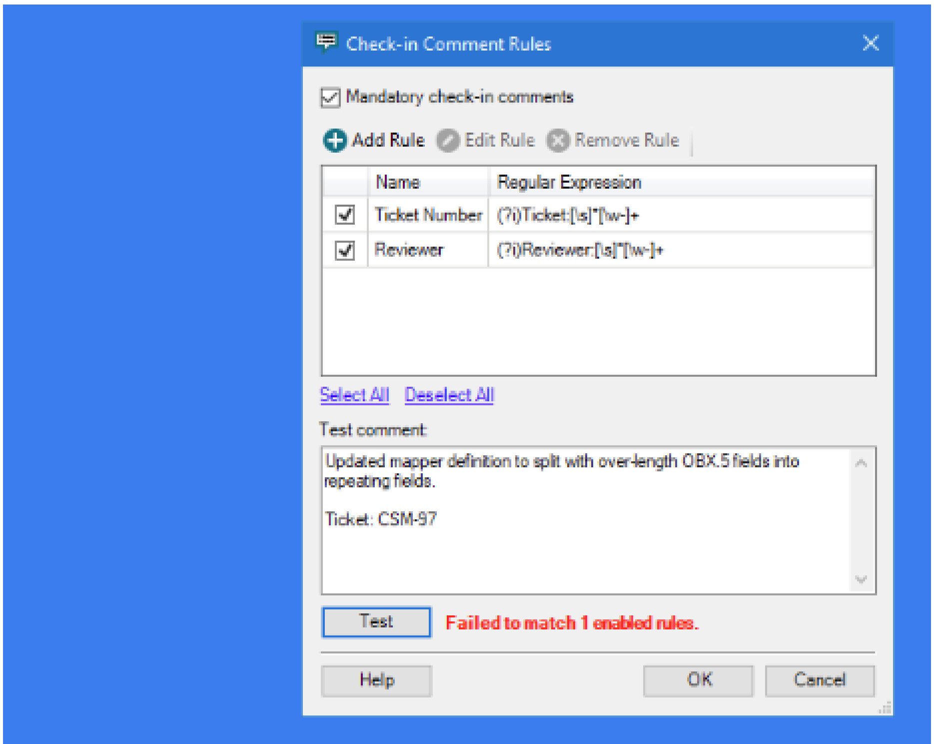
With both rules complete, your Check-in Comment Rules dialog should appear similar to the following:



Model answer for exercise

The rules for Ticket Number and Reviewer are similar, as the format of the two requirements is similar.

At this point, you should test a number of different comments to make sure the two rules work correctly with both correct and incorrect comments. When a test comment is entered that does not match all the rules, the failed rules will be highlighted.



A failed test comment with the failed rule highlighted

Issues and Notifications

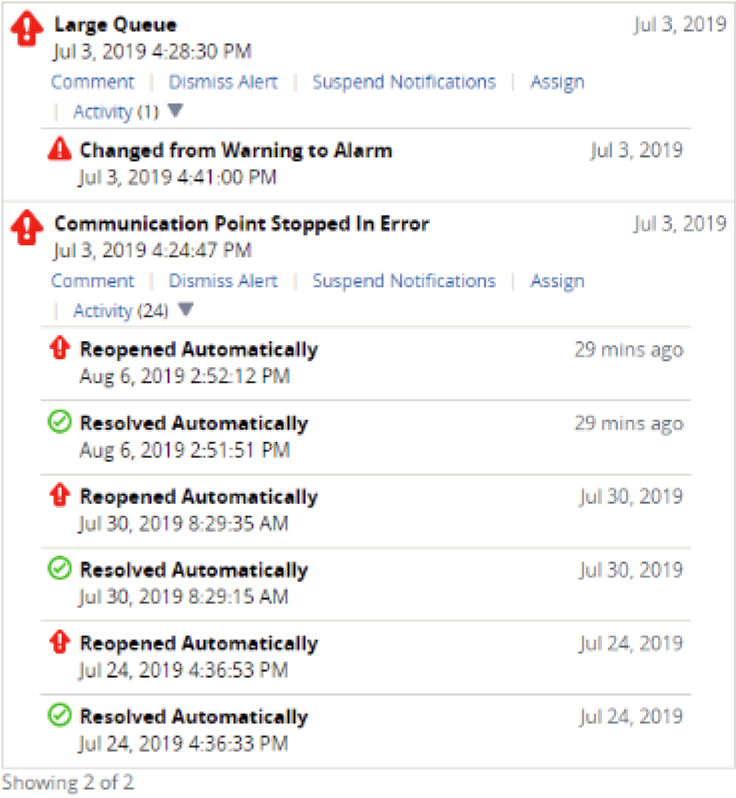
The Issue Management Framework

The Rhapsody engine provides a continuous monitoring framework that allows it to be managed by exception reporting.

The framework provides default trigger and threshold settings against which all components are monitored. When a setting is breached, the event is logged and an exception report is issued. The default trigger and threshold settings may be individually modified, and they may be further modified on a per component basis, allowing fine-tuning of the settings as required.

Exceptions are reported on the relevant component page in the **Management Console** and also by sending a notification report to a nominated group of users.

Each exception is handled as an event, initiated by the exception and remaining live until the exception condition has been resolved, either by user intervention or by the engine state returning to a level which permits the automatic resolution of the state.



The **Management Console** provides support for managing the event, notably by providing an event history (including user added comment) and by providing the ability to re-assign the responsibility for managing the event to another user.

The exception framework provides a three level hierarchy for notifications, primarily to ensure the appropriate resourcing can be applied to the resolution. Warning notifications are typically defined for events which are relatively harmless but need to be monitored. Alarms are defined for events which require prompt action, and escalation notifications are issued if an alarm remains unresolved for a nominated time period.

Management of the reporting hierarchy is simplified by the Watchlist functionality, which essentially groups components and the list of users to be notified if an exception occurs on any of the components attached to the list. The Watchlist structure provides the ability to change the notification list on a roster basis (including support for holidays), and also provides the ability for managing notifications when users are on leave or temporarily unavailable.

The Issue Event

An issue event is initiated when a threshold is exceeded or a trigger event occurs, and the following actions occur:

- An issue event is opened for the component; this is displayed and may be managed on the component detail page.
- Notifications are distributed to users subscribed to the component (either directly or by watchlist) by the selected communication methods.

Each event is raised at a level defined by the threshold specifications:

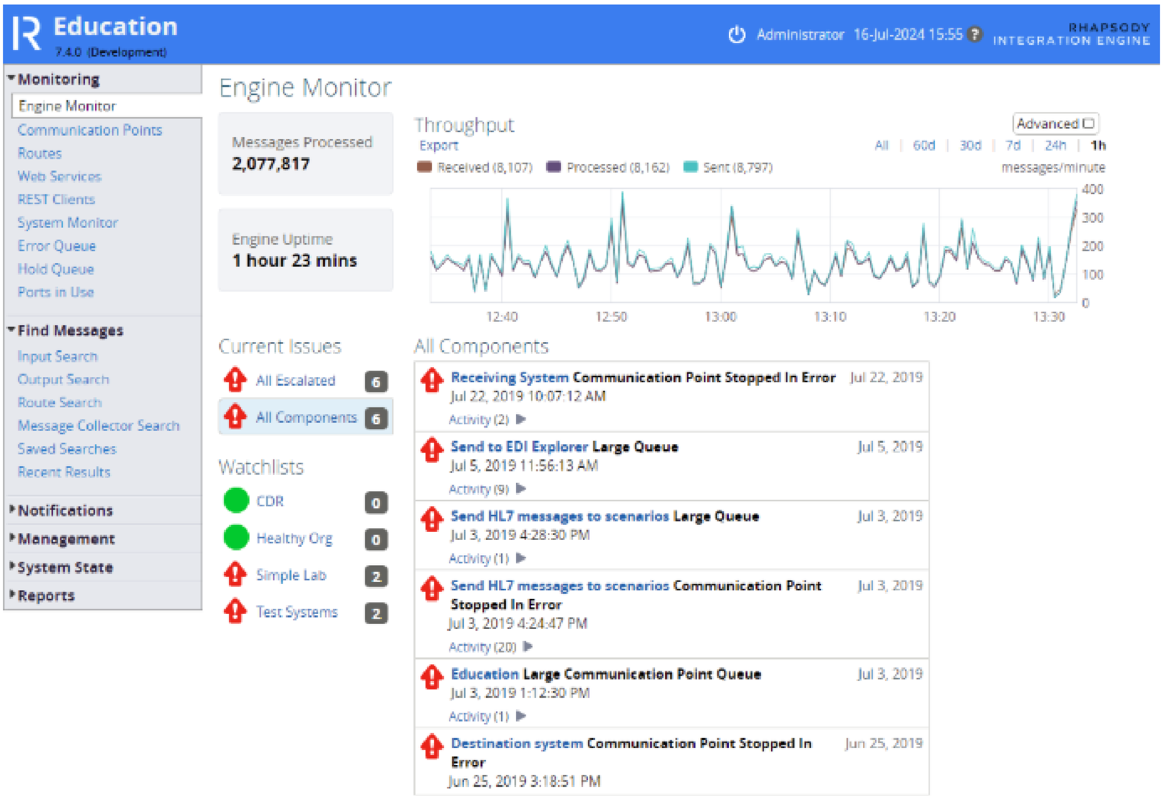
Warning - indicates an issue considered relatively harmless but which nevertheless requires attention.

Alarm - indicates an issue which requires immediate attention.

Escalation - indicates an issue raised in an Alarm state which has not been resolved in a defined time period; escalation raises the event state to the next level and triggers a further notification (typically to a more senior group able to ensure that action is taken).

The notification level is a cascading hierarchy; the level will be escalated if not resolved within the defined period.

An Issue summary is presented on the **Engine Monitor** page, a detailed listing of the Issue is presented on the component page, and if the component is included in a Watchlist, the Watchlist status is updated.



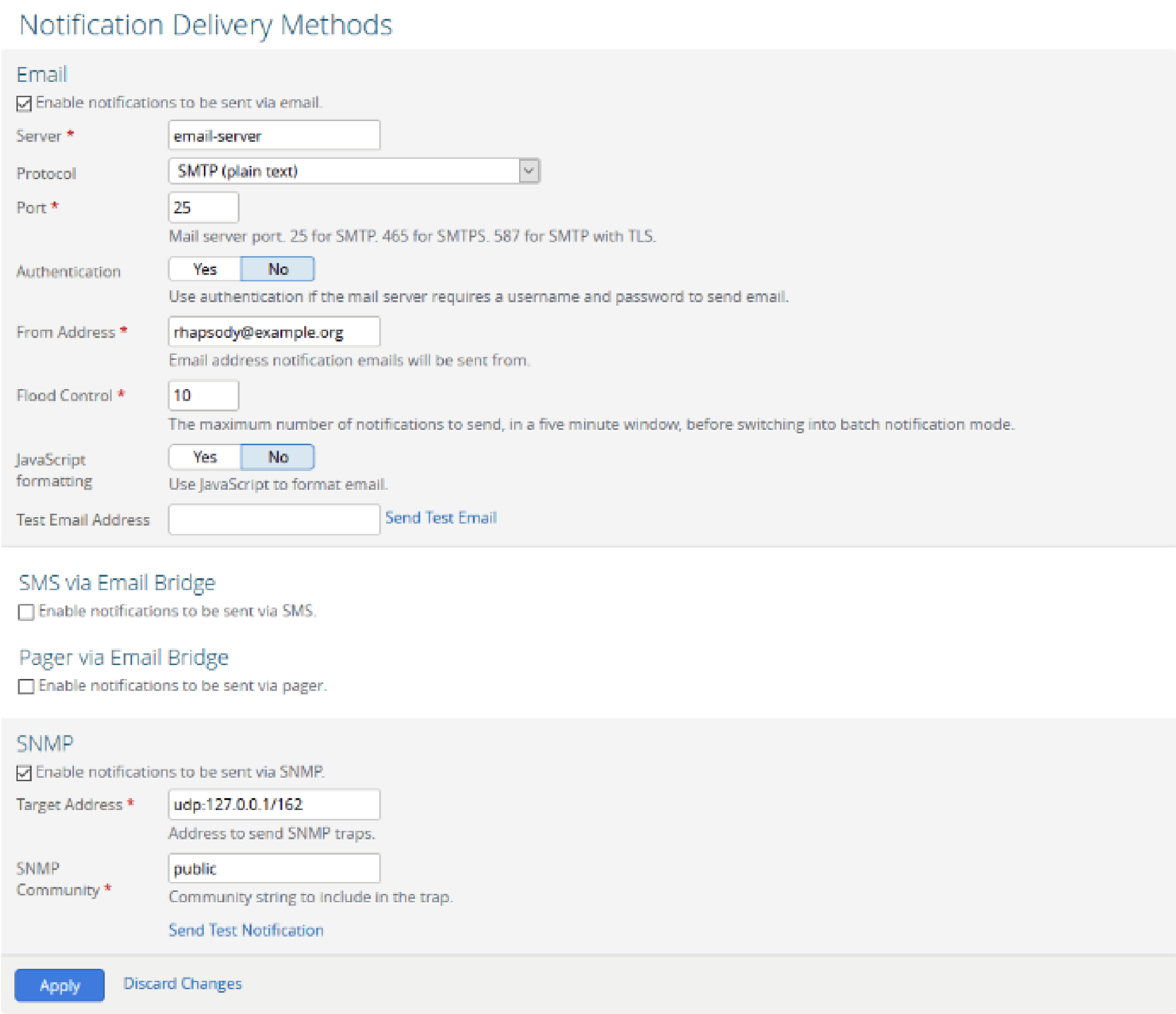
Engine Monitor showing issue and watchlist summary

Managing Notification Delivery

Notifications are sent to users using delivery methods managed by the system administrator. The Rhapsody environment supports communication by:

- Email
- SMS via an Email bridge
- Pager via an Email bridge
- SNMP (Simple Network Management Protocol)

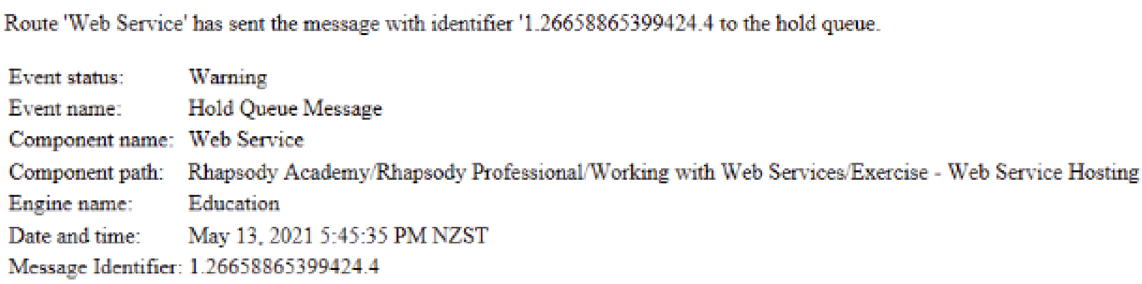
The delivery methods available for a system are managed from the **Notifications / Delivery Methods** page, which allows the appropriate methods to be selected and the relevant configuration to be defined.



Notification Delivery Methods

In this example, the administrator has selected Email and SNMP delivery methods, and provided the relevant configuration detail for each.

The default email format lists all the details relating to the notification in a table. You can edit the default email format by setting the **JavaScript formatting** option to a value of **Yes** and editing the contents of the JavaScript panel that appears. When the server generates a large number of notifications, the **Flood Control** setting is used to group multiple notifications into a single email message.



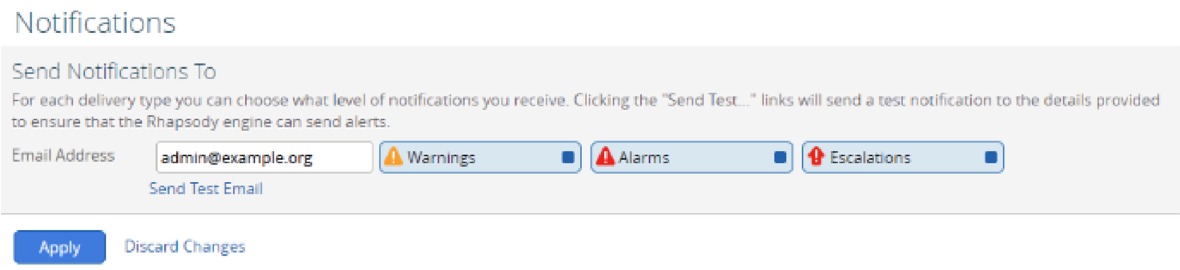
Route 'Web Service' has sent the message with identifier '1.266588653994244 to the hold queue.

Event status:	Warning
Event name:	Hold Queue Message
Component name:	Web Service
Component path:	Rhapsody Academy/Rhapsody Professional/Working with Web Services/Exercise - Web Service Hosting
Engine name:	Education
Date and time:	May 13, 2021 5:45:35 PM NZST
Message Identifier:	1.266588653994244

Example email notification with default formatting

Per user settings

Each user is required to provide the relevant personal details on the **Notifications / My Notifications** page. This allows the user to select the level of notification they will receive as well as define the user specific configuration (for example, if email delivery is selected, the user will need to supply their email address).



My Notifications settings

In this example, the user has defined their email address and selected Warning, Alarm and Escalation notifications for delivery.

Managing Thresholds - Default Settings

The Rhapsody engine provides a default threshold framework to ensure the operation and performance of all components is monitored by the Issue Management framework. The thresholds for individual components may then be fine-tuned as required.

The **Notifications / Default Settings** page permits management of the default thresholds, including:

- Defining the recipient for notifications for unmonitored components (that is, those not monitored by a watchlist)
- Resend and Escalation periods
- Default Threshold Settings

Default Notification Settings

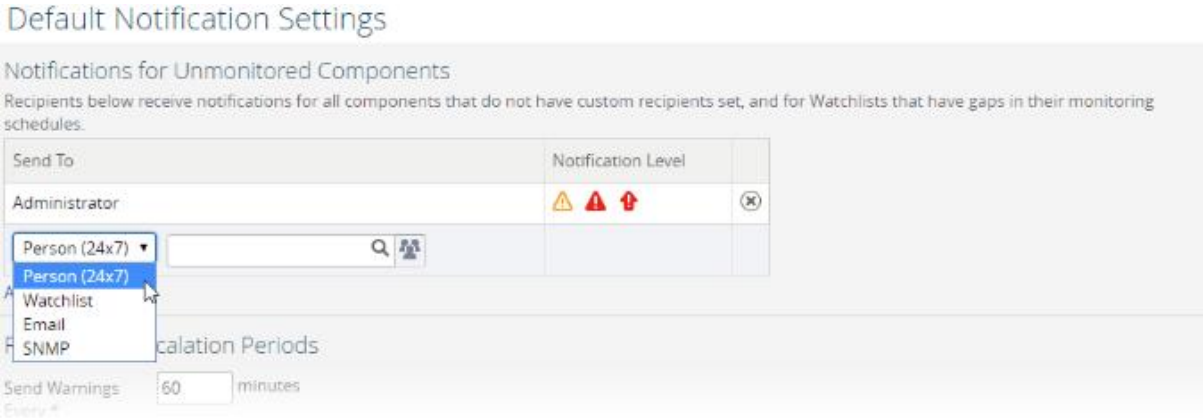
This section provides the settings for all components in the configuration which are not otherwise monitored by a watchlist. Configuration requires:

- Defining the recipient/s for notifications
- Establishing the Resend and Escalation periods

Notifications for Unmonitored Components

The Recipient section allows selection of the recipients for notifications and provides support for selecting recipients who are not configured as users for the Rhapsody system. The recipients may be selected from:

- **Person (24x7)**: allows locating a Rhapsody user by typing a partial name into the search box or by selection from the list of users.
- **Watchlist**: available if watchlists have been defined in the system, allowing all the recipients for that watchlist to be included, simplifying maintenance of the default recipient list.
- **Email**: allows notifications to be sent to users who aren't defined as Rhapsody users; simply enter the email address.



Setting the delivery method for a new default recipient

Resend and Escalation Periods

These settings control how often the notifications are resent and the length of time to wait before the event level is escalated. In normal use, it is anticipated that the users receiving notifications for each level will overlap but will include additional recipients as the level rises to ensure the event is not ignored.

Multiple changes may be made to the Notifications Settings at any time. Select **Apply** to save and apply the changes to the engine.

Managing Thresholds - Default Settings

Default Threshold Settings

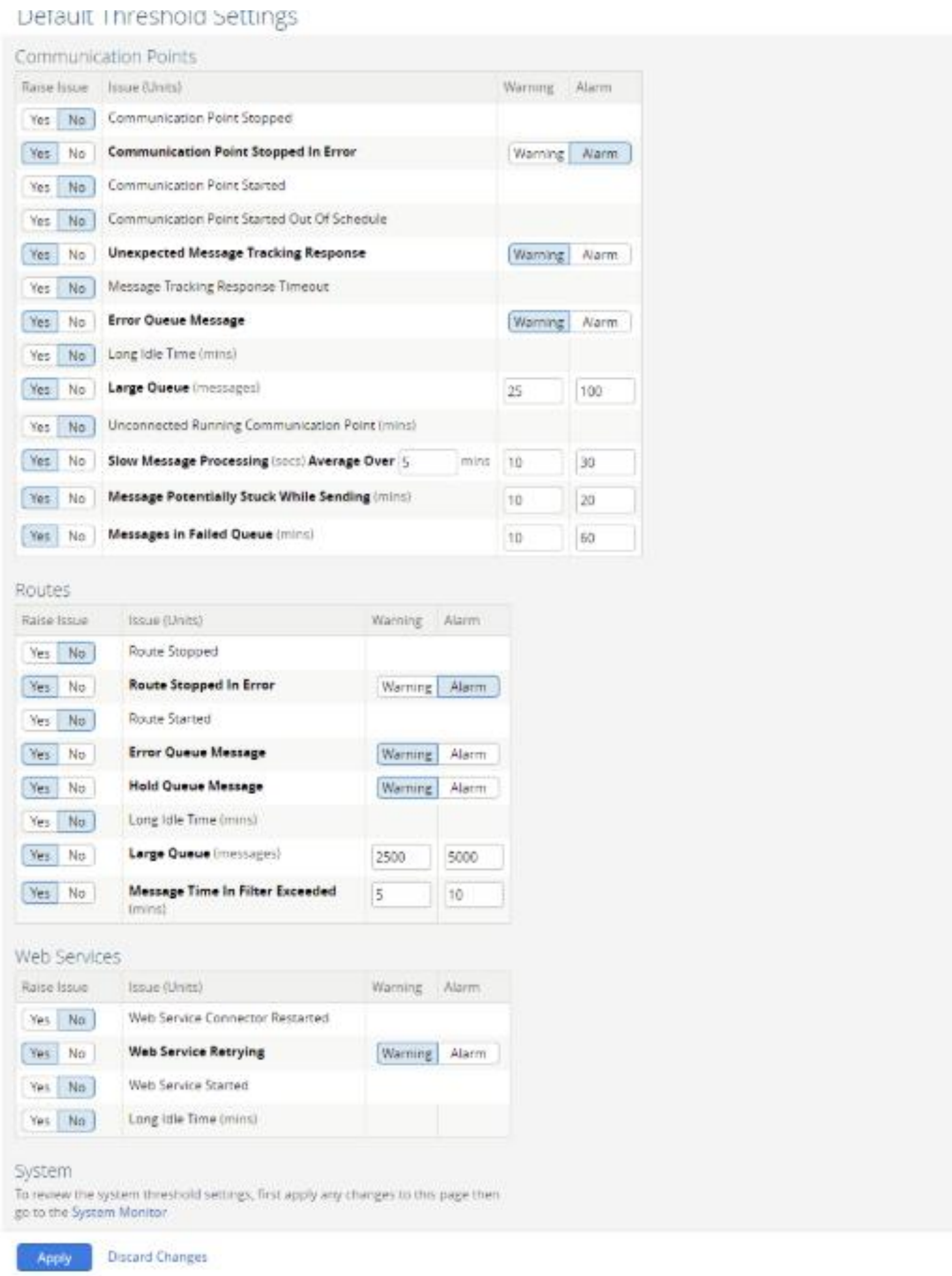
The **Default Thresholds** settings define the trigger levels and thresholds for all components in the engine without custom notification settings. They also provide the ability to define the level at which the alert will be raised if a notification is issued.

Choosing appropriate default values is important for engine management; too low and there will be too many notifications issued and too high, and notifications are less likely to highlight issues promptly. Since these settings are system-wide, it is generally best to use widely applicable values, with custom levels for components needing more specific settings. For most sites, the default values set at installation time are appropriate.

The items provide either a threshold or a trigger function:

- A trigger event (for example, component stopped, component stopped in error); if enabled, the user selects the level at which to raise the event.
- A threshold (for example, queue exceeds 20 messages); if enabled, the user selects the threshold at which a warning is issued, and the threshold at which an alarm is issued.

Changes will be saved and applied to the engine when the **Apply** button is selected.



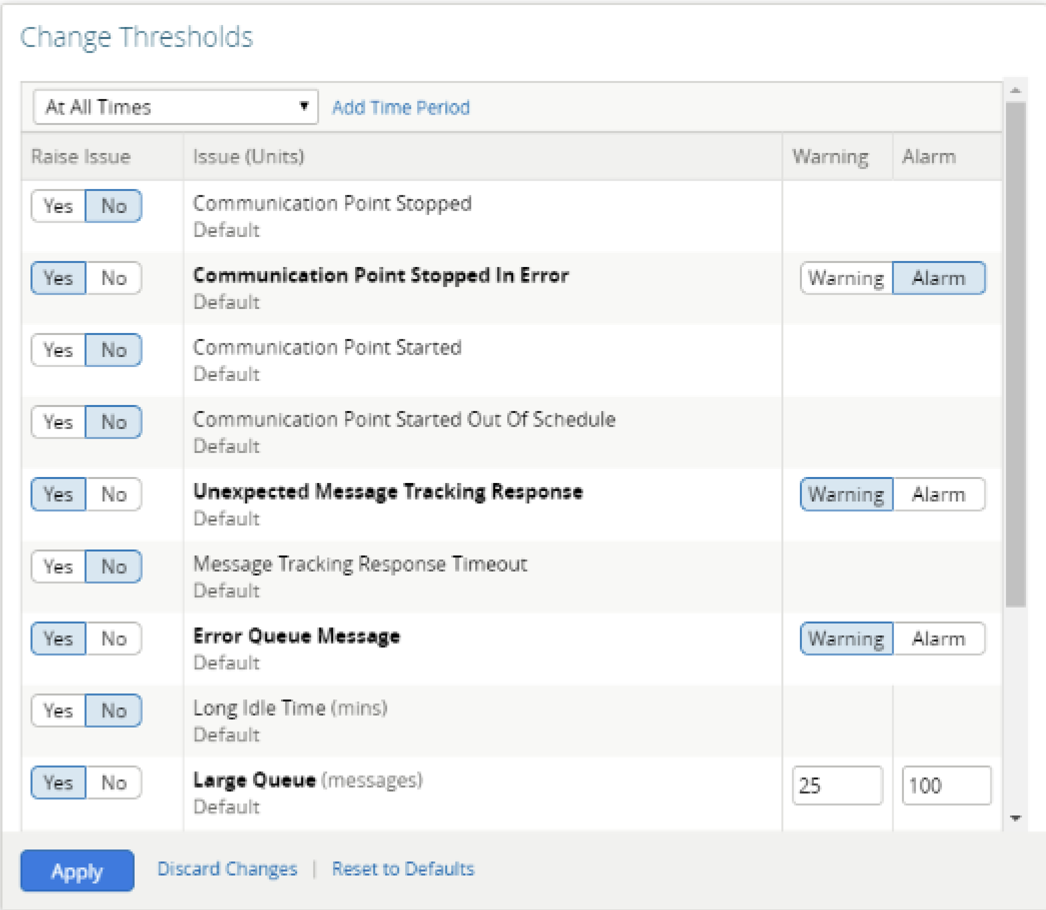
Default Threshold Settings

Component Thresholds

The **Custom Thresholds** link on the component details page provides the ability to modify threshold settings for each component, overriding the default settings for that component.

The custom thresholds may be established for only a portion of the day, for example, to ensure that more stringent limits are applied during the working day, or to relax limits for a period to accommodate overnight batch processing.

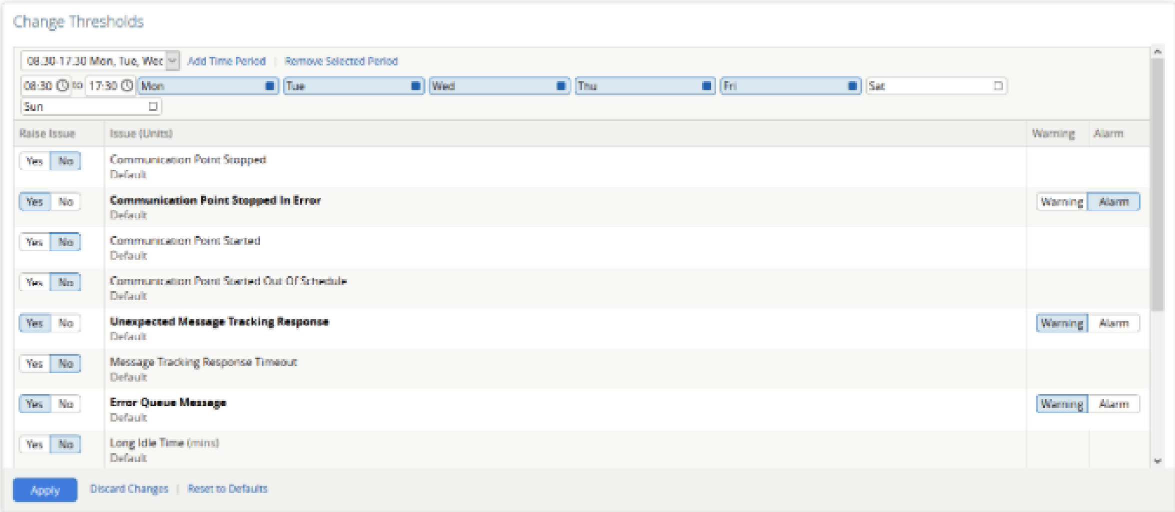
Once all changes have been made, select **Apply** to save the settings and update the engine configuration.



Custom Thresholds for a communication point

To define a time period, select **Add Time Period**. This will open a dialog allowing the period definition to be created. The start and end time for the period should be defined by typing the desired values into the relevant box (these will be displayed in HH:MM format) and by selecting the days for which the time period should be applied.

Select the desired time period, adjust the threshold settings, then select **Apply** to save the changes. Note that time period definition may be modified by selecting the period and adjusting the time details prior to saving.



Custom Thresholds defined for a particular time period

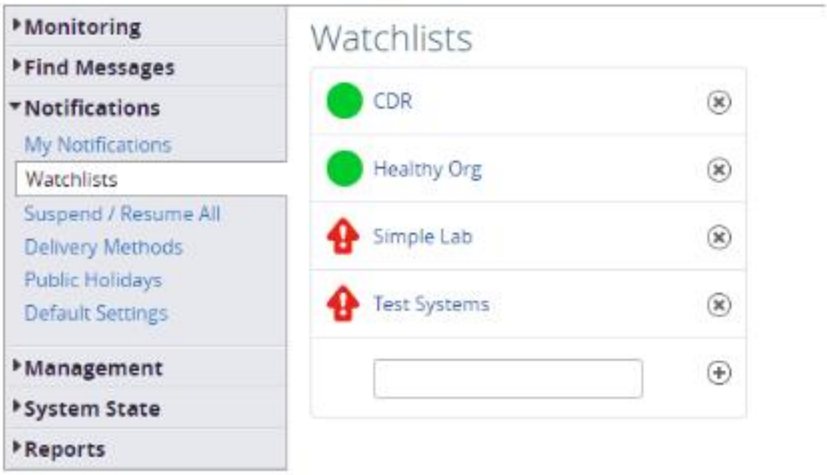
Watchlists

Watchlists provide the ability to define a group of users who will receive notifications for a group of components. Each user will receive notifications for the event states which they have defined in their individual settings.

The administrator is able to define a schedule to manage notifications, permitting, for example, one group of users to receive notifications during the work day, and a different group to receive notifications outside of normal working hours. By default, a single 24x7 schedule is created ensuring there are no gaps in the coverage, but the administrator is able to define up to 3 schedule groups for the watchlist.

The **Notifications / Watchlists** page provides access to the list of watchlists configured for the engine, displaying

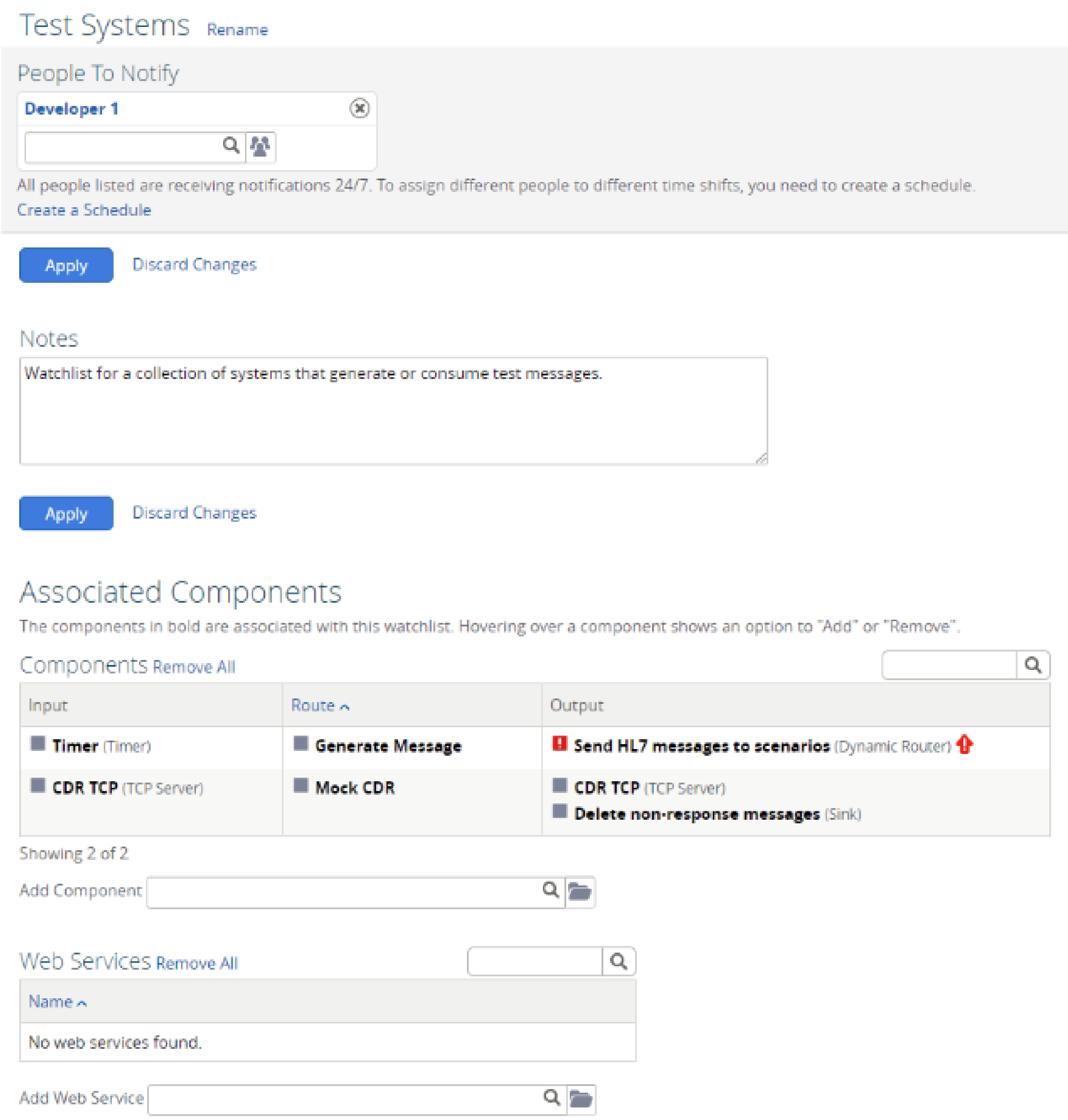
- a list of the watchlists with their current state
- a clickable name to access each watchlist
- a **Remove** (x) icon to remove a watchlist
- a text box with an **Add** (+) icon to create a new watchlist



A list of watchlists on a Rhapsody Engine

Building and Managing a Watchlist

Each watchlist may be managed by selecting its link on the **Notifications / Watchlists** page.



Configuration for a watchlist

The page provides management of the:

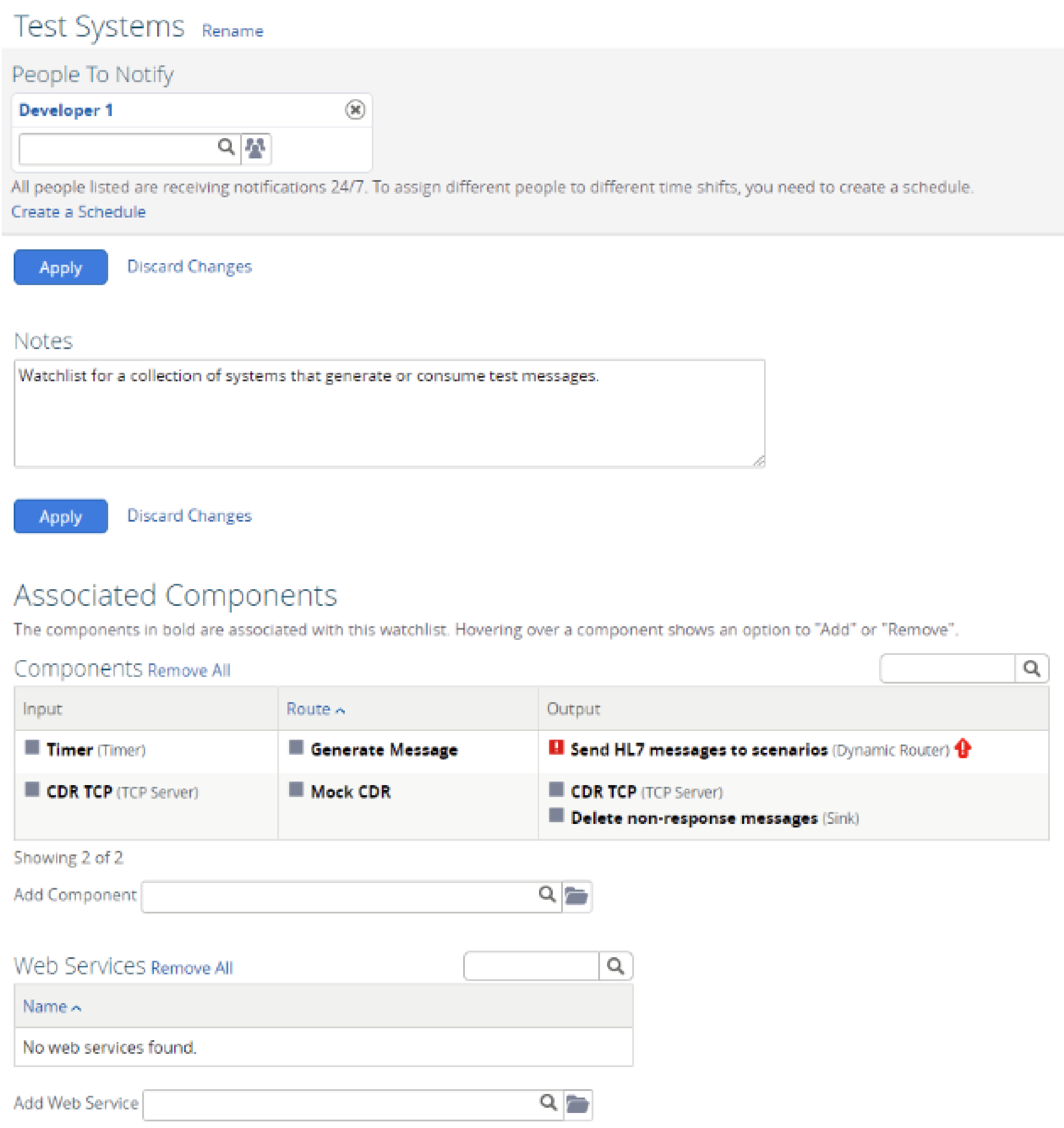
- Recipient list for notifications (People to Notify)
- Notes that describe the purpose of the watchlist
- Components associated with the list

Users may be added to the list by typing a partial name in the users text box; a list of matching names will be displayed. They may also be added by selecting the names from a list by selecting **Users** to the right of the text box. Note, only one user may be selected at a time. Once the user has been identified, select the **Add** icon. Users may be removed from the list by selecting the **Remove** icon next to the name. Once all the user changes have been made, select **Apply** to save the changes and apply them to the engine configuration.

The default configuration provides for a single group of users to be applied on a 24x7 basis. If desired, a more fine-grained approach may be applied by creating up to 3 user schedules to be applied based on the time.

Components may be added to the watchlist by typing a partial name into the text box in the Associated Components section of the page. A list of all matches will be displayed, allowing selection of the desired component. Alternatively, select the folder icon to display a list of all components for selection. Note that only the selected component will be added, although associated components and their current state will be displayed.

For example, in the view below, the route **Register patient** is included in the watchlist (displayed in bold) but the communication points are not included.



A watchlist that includes a route but not the connected communication points

Components are automatically added to the watchlist on selection and no explicit Apply action is required.

Watch List Notifier

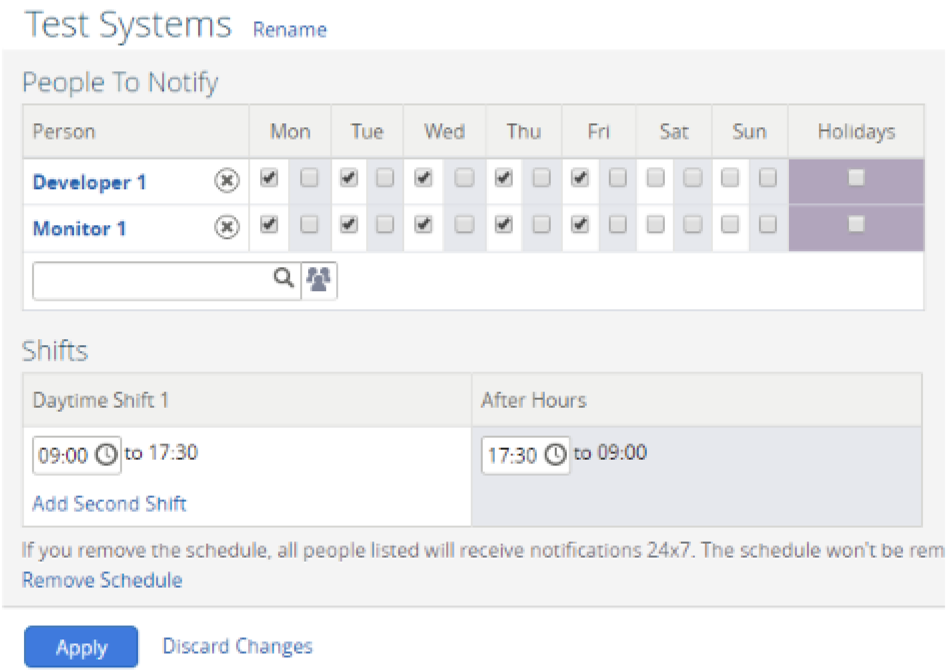
The Watch List Notifier (discussed in the Communicating with Rhapsody module) can be configured to listen for events sent to one or more watchlists. This is useful when there is a need to process certain Watchlist events in Rhapsody.

Building a Watchlist Schedule

The default schedule for a Watchlist is applied on a 24x7 basis. It may be fine tuned by adding shifts to the day.

Shifts provides the ability to build a notifications roster based on two shifts per day, with the option of creating a third shift. This functionality is accessed by selecting the **Create a Schedule** link on the **Watchlist Management** page.

The default shifts (Daytime and After Hours) will then be created and displayed, and the current recipients will be added to the Daytime shift.



Configuration of shifts for a watchlist

Recipient Management

The roster displayed for each recipient is color coded to match the shifts displayed in the **Shifts** section of the page. Note that an additional roster is available by selecting the **Holidays** item as a special case; this provides for alternative cover for public holidays (defined and managed under the **Notifications / Public Holidays** menu item).

Select the shift allocations for the recipient by selecting the appropriate checkboxes (selection denoted by the tick mark in the box).

Once all changes are complete, select **Apply** to save the changes and update the engine configuration.

Note that each user must ensure that the appropriate details are entered for the delivery mechanisms they use (**Notifications / My Notifications**). The Administrator may view and update these settings by selecting the user name link; this opens the **Notification** details page for the user and enables the administrator to ensure that the settings have been correctly defined.

Shift Management

The default shifts created are:

- Daytime Shift 1: 09:00 to 17:30
- After Hours: 17:30 to 09:00.

The shift structure may be manipulated by modifying the start time for the shift (the end time is fixed against the start time of the following shift). For example, to modify the Daytime shift to terminate at 18:00, enter the new start time for the After Hours shift; once this value has been entered, both shift entries will be updated appropriately.

Once all changes are complete, select **Apply** to save the changes and update the engine configuration.

Adding a second Daytime shift

If desired, a second Daytime shift may be added by selecting the **Add a Second Shift** link. This will create the Daytime Shift 2 by splitting the existing Daytime Shift 1 period in half. A third checkbox will be added for each recipient to allow allocating resources to the new shift.

Modify the start times for each shift as desired, and select **Apply** to save the configuration and apply it to the engine.

Adding recipients after the creation of a second daytime shift will automatically select both daytime shifts for the user to receive notifications.

Shift Coverage

The watchlist management page provides the ability to assign users to receive notifications based on the shift coverage.

It is not necessary to ensure that the watchlist fully defines the coverage; any shift without coverage will fall back to the default notification structure for the engine to ensure that all components are continuously monitored.

Watchlists and Public Holidays

The **Notifications / Public Holidays** page permits definition of the coverage for the **Holidays** notification schedule. This overrides the normal schedule if selected in the Recipient shift allocations.

Public Holidays may be added either manually or by importing an XML file defining the structure.

Manual entry requires:

- Adding a label for the Holiday (for example, Christmas Day).
- Defining the date, either by manual entry (dd-mmm-yyyy, for example, 25-Dec-2023) or by selecting the date from the calendar.
- Adding the entry to the list by selecting **Add**.

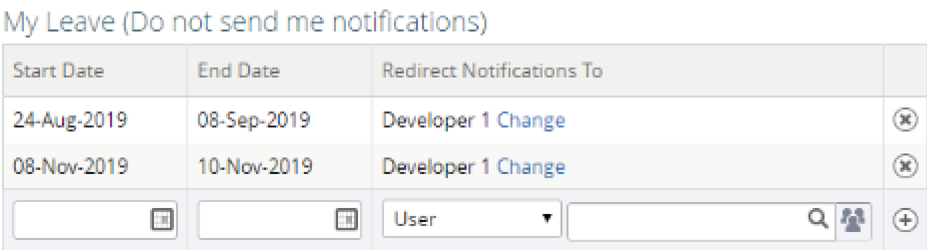
Once all modifications have been made, save the changes and apply them to the engine configuration by selecting **Apply**.

Watchlists and User Vacation Schedules

Each user may manage a personal vacation schedule to define periods when they are not available to receive notifications; for example, when out of the office for an extended period or on vacation.

The user may choose to not receive notifications during the defined periods, or to redirect the notifications to another user or email address.

The vacation support is available in the **My Leave** section of the **Notifications / My Notifications** page for the user.



Leave dates entered for a user

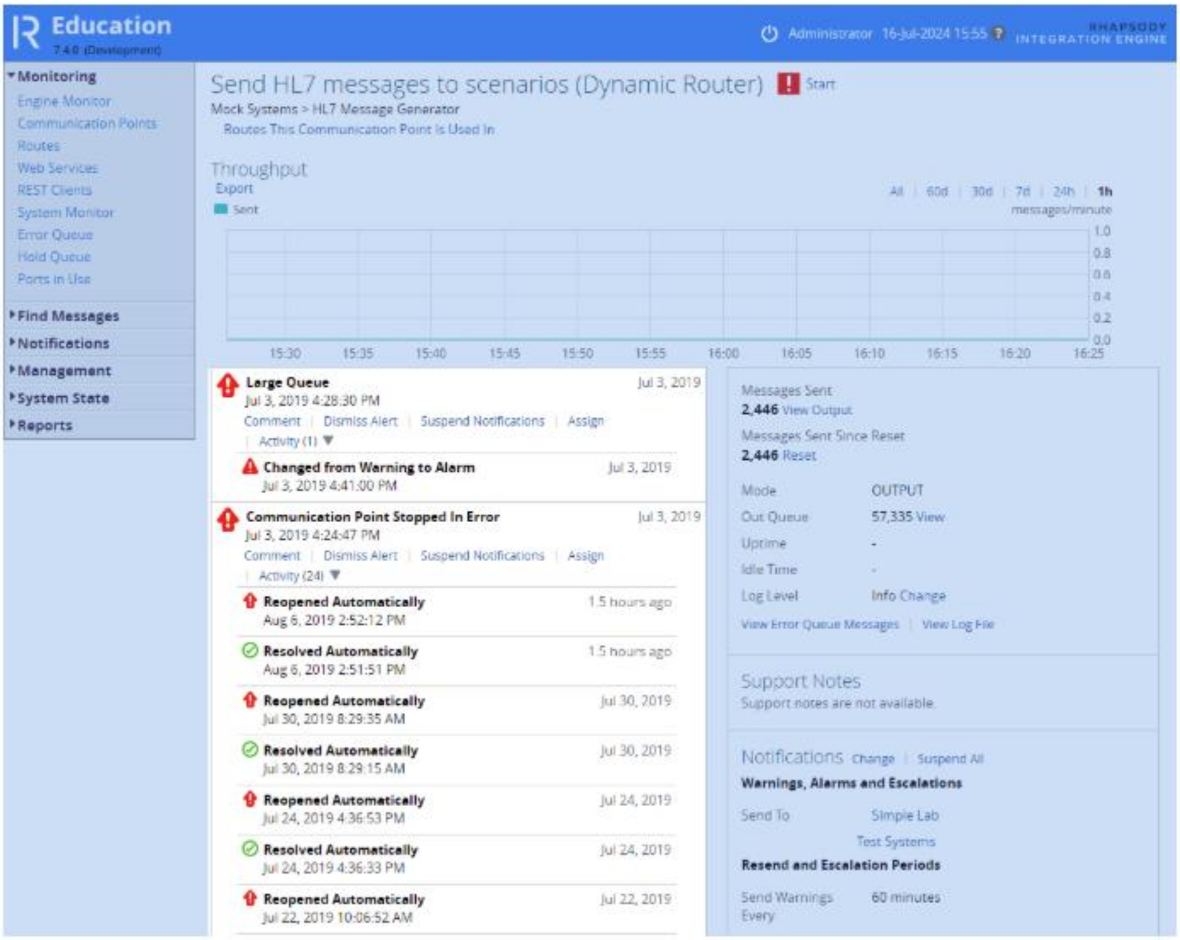
The vacation period is defined by entering the start and end dates (either manually or by selecting using the calendar tool) and defining the redirection action to be taken.

The changes take effect immediately after the entry is added to the **My Leave** list.

Managing Events

The component detail page provides information on open issues as well as the log information for the component. Both are presented as activity logs, sorted in time descending order.

The issue activity component, located immediately below the throughput graph, presents the detail of any issue currently active which is affecting the component.



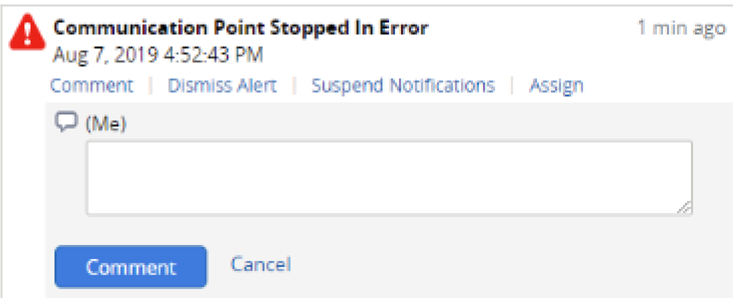
The list of notifications for a communication point

The activity component:

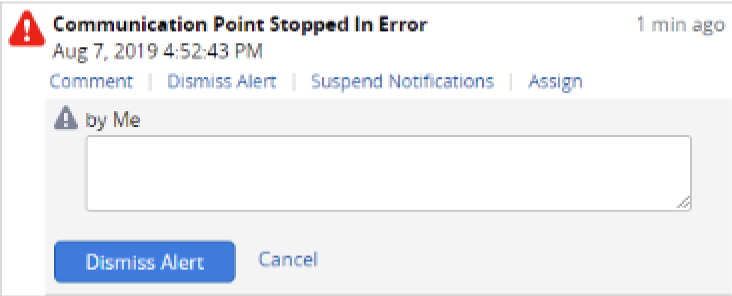
- identifies the issue/s currently open
- displays the event status level
- presents the activity stream for the event/s

In addition, it allows the user to:

- Record a **Comment**, for example documenting the cause of the issue or actions taken to identify the cause of the issue.

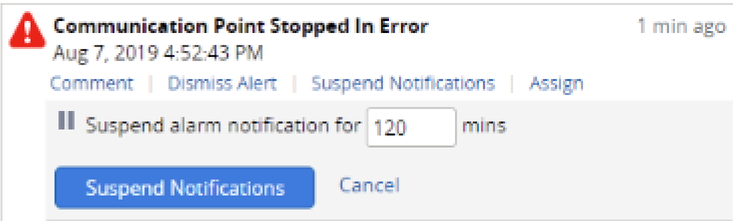


- Dismiss or Re-open an alert (for example; the event might be dismissed when there is a known cause external to Rhapsody).

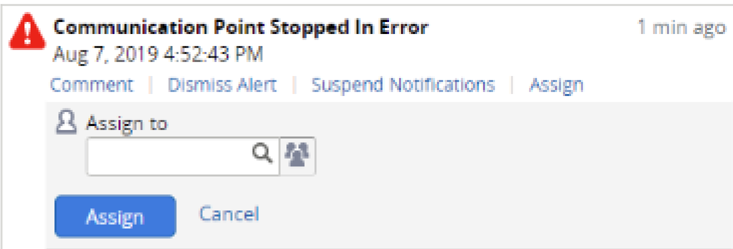


Once an issue is dismissed, the Issue status for both the component and the Watchlists to which the component is assigned will return to **No Warnings or Alarms**. The dismissal comment will be added to the activity stream for the component and is accessible from the Activity Log for the component (displayed below the Issue detail).

- **Suspend** or Resume Notifications.



- **Assign** the responsibility for managing the issue to another user (for example, at the end of a shift).



- Hide or Open the **Activity** list.

The Event Lifecycle

The issue management functionality of the Event Activity list effectively provides a trouble ticket to manage the event until it is closed. Closure may occur automatically if the threshold condition is no longer met, or if the issue is dismissed by the user.

Activity	Comment
Threshold value exceeded or Trigger Event occurs	Issue Event initiated Notification issued at the alert level identified by custom (component level) settings or by default engine wide setting Watchlist updated to show active issue Component detail page identifies issue Issue event logged to system log
User accesses component details page	Issue review
User adds comment to event history	Optional step, typically indicating action being taken or identifying some feature of the issue
Behind the scenes activity to identify and resolve cause	
Assignment of issue to a user	Issue assigned to a user with specialist knowledge or to transfer responsibility (for example, at the end of a shift) Issue included in the **Assigned To Me** list of current issues displayed on the **Engine Monitor** page
User suspends notifications; comment added to identify reason	Cause may be external or may take some time to resolve so notifications are suspended for a period to reduce the traffic
User resolves issue User dismisses issue	Once the issue is resolved, the user dismisses the issue, potentially adding a comment to the activity stream, particularly if the cause was obscure (this will aid future resolution if it recurs) User may elect to suspend notifications for a period if it is likely that the issue may sporadically recur

Activity	Comment
Issue re-opened automatically by system	The engine continuously monitors the state of the component If the issue recurs within a reasonably short time-frame, the issue will be re-opened If it recurs in the future, a new event will be raised

Exercise - Issues and Notifications

Overview

In this exercise, we will work through the configuration and management of the Notifications framework and the management of issue events.

The key elements of the framework are:

- Delivery Methods: how the system communicates Notifications.
- My Notifications: how the Notifications are delivered to each user using the Delivery Mechanisms; typically managed by the user.
- Default Settings: the default threshold and trigger settings applied to all components (which may be overridden at the component level).
- Component Thresholds: specific threshold and trigger settings applied to an individual component to better reflect its operational character.
- Watchlists: the association of a group of users to be notified with a set of components to be monitored.
- Watchlist schedules: the rules for each watchlist which determine the fine detail of who will be notified at any time of the day.

Setup

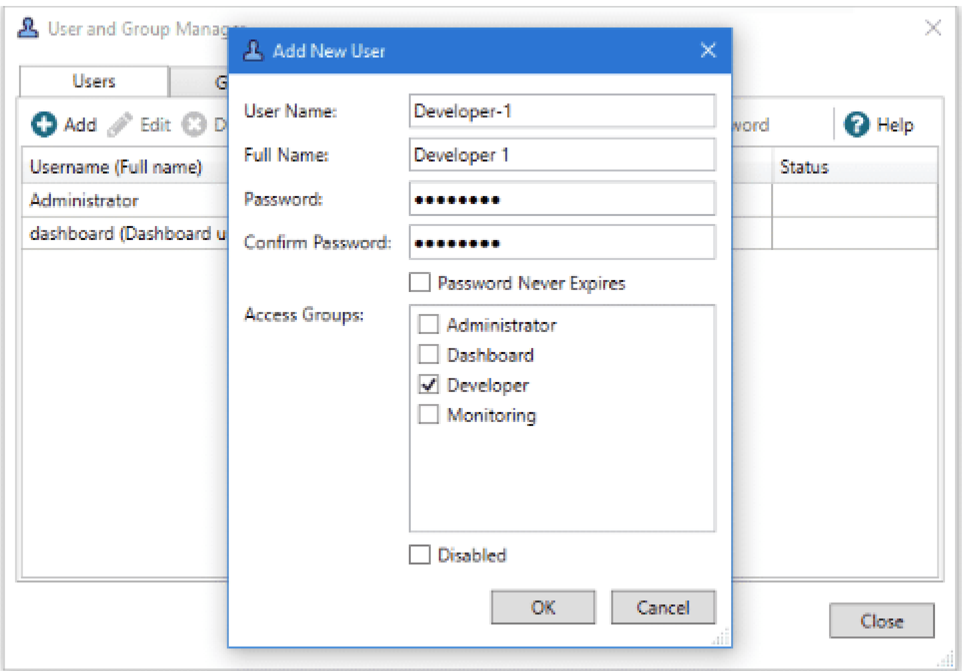
Delivery Method

This exercise uses the Email delivery method to send notifications to the users. Before starting this exercise, you will need to determine the following connection details:

ⓘ If you don't have access to an email server to receive the notification messages, you can use one of the suggested test email servers mentioned in the E-Mail Client exercise in "Communicating with Rhapsody".

- Mail **Server**: host name or IP address of the email server.
- **Protocol**: typically SMTP (plain text) but you will need to verify whether an encrypted connection is needed.
- **Port**: which port to use to connect to the server to send email.
- **Authentication** required: whether the server requires the connection to be authenticated before it accepts email messages.
- **From address**: the address the email is to be sent from; this should be a real address which is monitored in case any emails cannot be delivered and are returned.
- **Flood control**: minimizes the impact of high volumes of notifications if large numbers are issued over a short period of time to prevent flooding of the mail server.

- **Users**
- The Watchlist component of the exercise requires multiple users to use when setting up a schedule.
- These should be set up using the **User Manager** in the IDE (**View / User Manager** or the Utility icon on the icon bar which provides access to the User Manager).
- Open the **User Manager** and select **Add** to add a new user.



The **Add New User** dialog will present the items to be filled in.

For the purposes of the exercise, ensure you have the following users created:

Username	Access Group
Administrators	
Developer-1	Developers
Developer-2	Developers
Monitoring-1	Monitoring
Monitoring-2	Monitoring

1. Delivery Methods

The first step in configuring Notifications is to define the details for the Delivery Methods to be used for your site. Configure the methods as follows:

1. 1

Open the **Delivery Methods** page in the Management Console (note that you will need Administrator rights to access this page).

2. 2

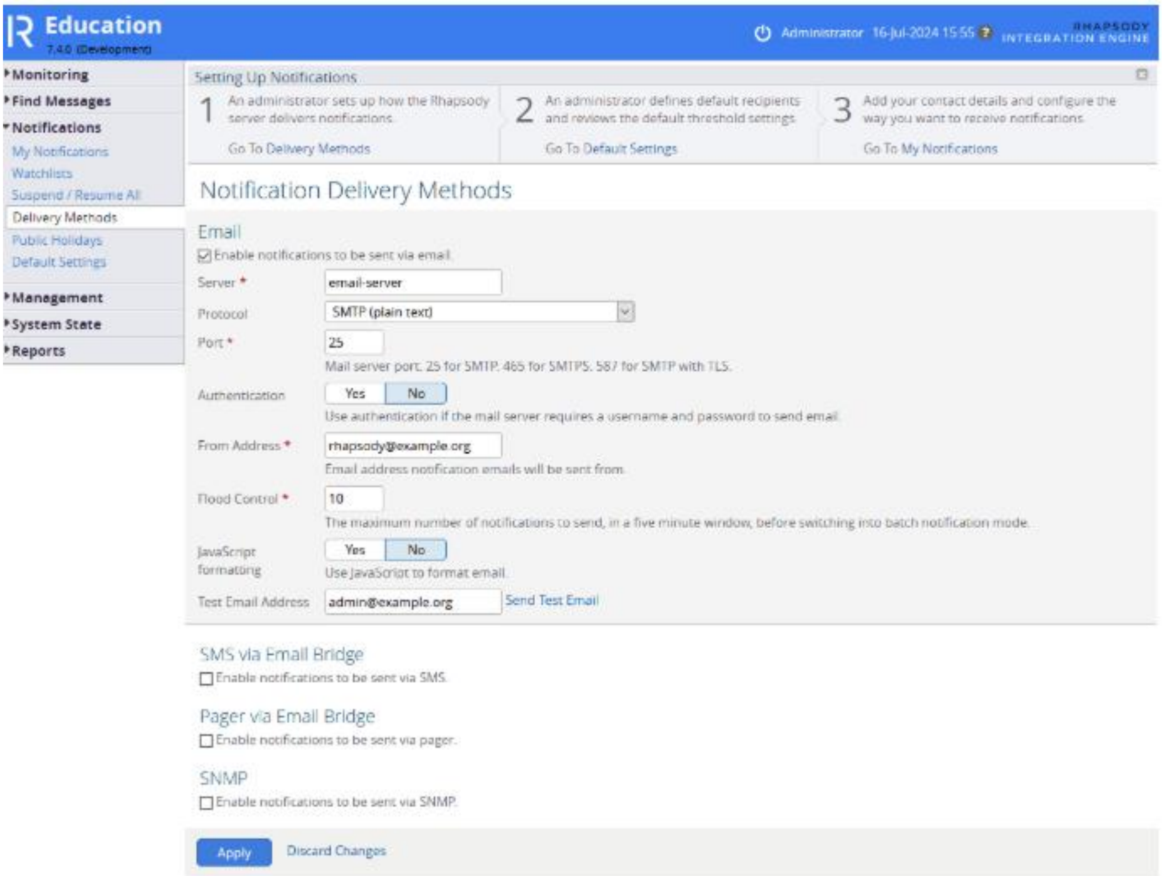
Ensure that the **Email** method is selected.

3. 3

Fill in the required boxes (highlighted with a red asterisk) using the information for the email server you are using.

4. 4

Select **Apply** to save and update the engine with the details.



Setting up the email notification delivery settings in the Rhapsody Management Console

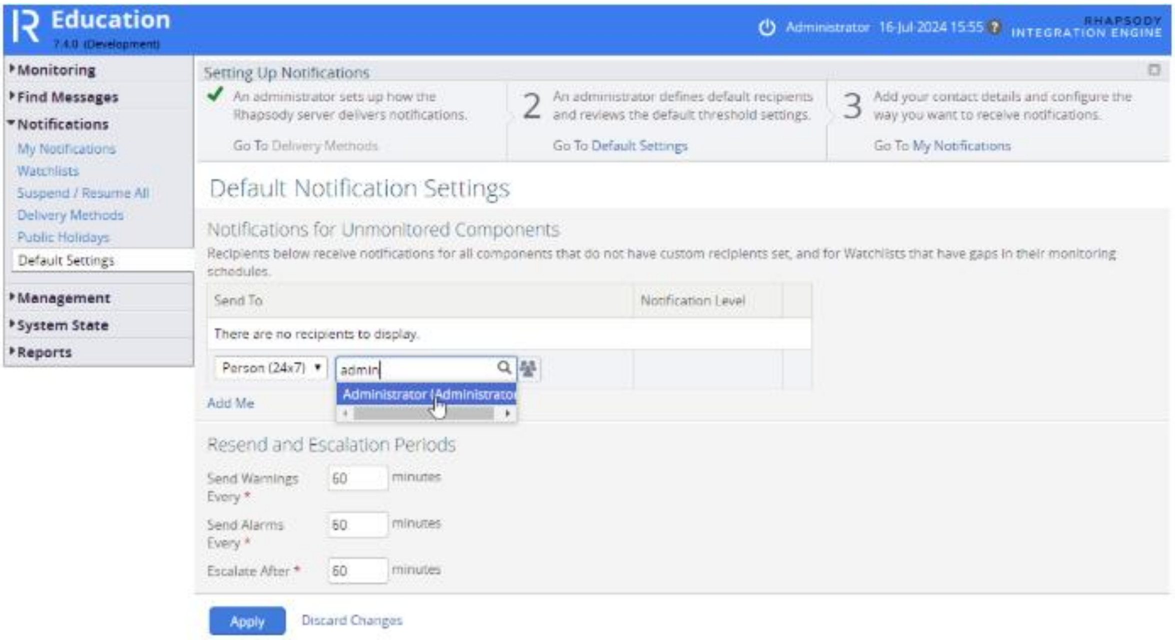
Once you have filled in the details, send a test message to a valid email address to confirm that the settings are correct.

Once you select Apply, a tick will appear alongside the first item in the Setting Up Notifications panel at the top of the page. This indicates this step is now complete and that we can move on to the next setup step.

2. Default Settings

Next, open the Default Settings page to define the default notification recipients for unmonitored components.

Configure the Recipients



Selecting the administrator as the default recipient for unmonitored components

1. 1

Ensure that the **Person (24x7)** category is selected in the drop-down box.

2. 2

Type part of the user name in the text box; the available options will be displayed.

3. 3

Select the required name.

4. 4

The user will now be added to the list.

5. 5

Once all the users have been added, select **Apply** to save the list and update the engine.

You will likely see an error message at the top of the page at this point. We will correct this in a later setup step.

Resend and Escalation Periods

If a notification is issued, but the issue isn't rectified within a defined time frame, the notification will be re-issued to ensure that it hasn't been overlooked. Additionally, if the issue remains open, the alert level of the event will be raised and Escalation

Notifications will be issued. Typically, these are sent to a more senior user group to ensure that appropriate action is taken.

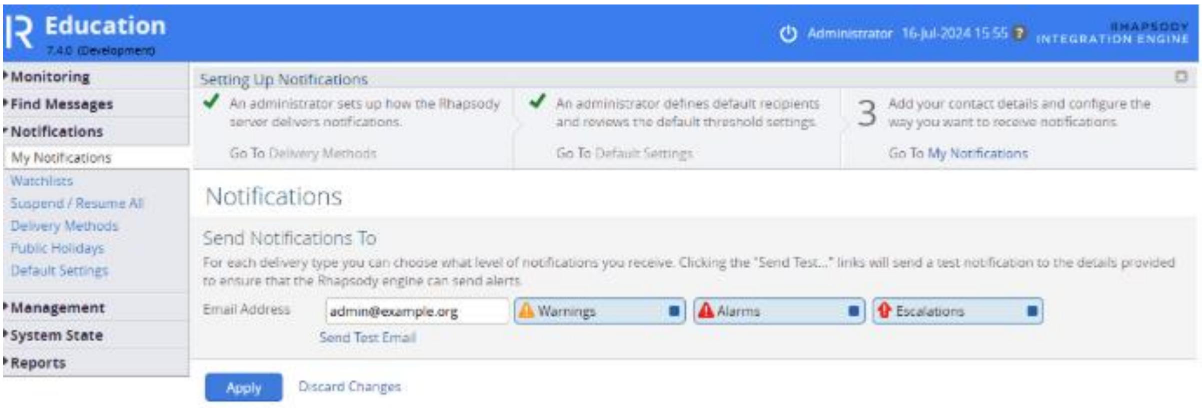
For the purposes of this exercise, you may wish to set these options to low values as you investigate the notifications functionality.

Select the **Apply** button to save the settings and update the engine.

The settings on the Default Settings page apply system-wide, but may be overridden by custom settings on a component.

3. My Notifications

Finally, to complete the notifications setup, we need to specify the email address associated with the administrator user. This is the user configured to receive notifications for all unmonitored components. Open the **My Notifications** page and enter your email address into the box provided, then select **Apply** to save the changes. Select **Send Test Email** to check that everything is working correctly.



Setting up email settings for Administrator user

Once an email address has been set on this page, the **Setting Up Notifications** panel at the top of the page will disappear. This tells us that the initial notifications setup is complete.

Exercise Resources

Now that we have set up notifications, we can import a configuration that will generate various notifications. The exercise depends on three routes:

- **Error Generator** - Notification triggered by a message going to the Error Queue at regular intervals.
- **FIFO** - Continually generates a slow processing notification.
- **Transient load** - Generates a slow processing notification only when messages are being loaded.

- **Resources**
- Download the RLC and import the configuration to your Rhapsody environment. Make sure all routes are started before continuing.
- The three routes are available here Exercise - Issues and Notifications.rlc.
- # Trigger Event
- When a trigger event occurs, it will open an Issue for the relevant component and send the appropriate Notification. As the event is typically a one-off or relatively rare event, the Issue may be investigated and the Issue dismissed.
- **Example - Error Generator**
- The **Error Generator** route utilizes a Timer to generate a message every 5 minutes, and the route contains a misconfigured JavaScript filter which causes an error to occur during processing. As there is no connection attached to the Error Connector of the filter, the message is placed into the Error Queue.

1. Ensure that the **Route Error Queue Message** is enabled at the Warning level in the Default Settings (make sure that you **Apply** any changes needed).
2. 2

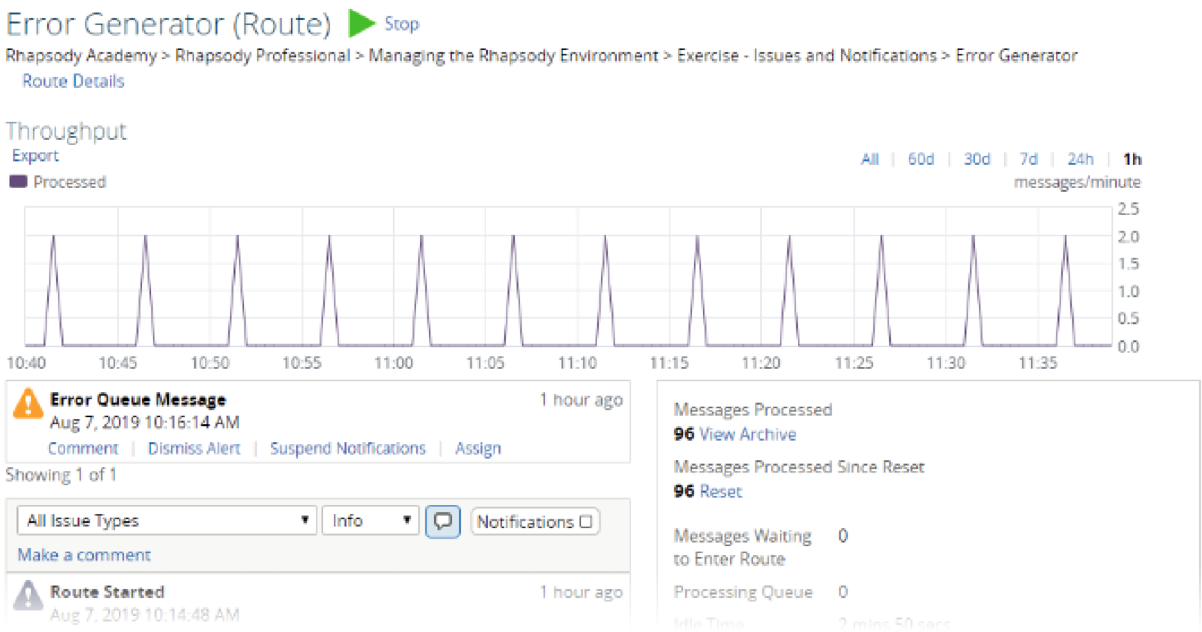
Start the Output communication point, Route and the Input communication point for the **Error Generator** route.

3. 3

Open the **Engine Monitor** page; at the next page refresh, the warning for the route will appear. Select the link to take you to the **Route Details** page for the component.

4. 4

As the Issue is triggered each time a message is sent to the Error Queue, **Dismiss** the issue and monitor the page until the next message is input to the route; the Issue will be raised again.



Error Generator Route in Management Console showing Error Queue notification

Threshold - Transient

The route **Load Patient Details** may be used to illustrate the effect of a transient load by providing a large number of messages to the Input communication point. These will be processed rapidly by the route, but queued in the Output communication point.

The load on the Output communication point will be transient, and the Issue will resolve automatically once the load drops below the threshold level.

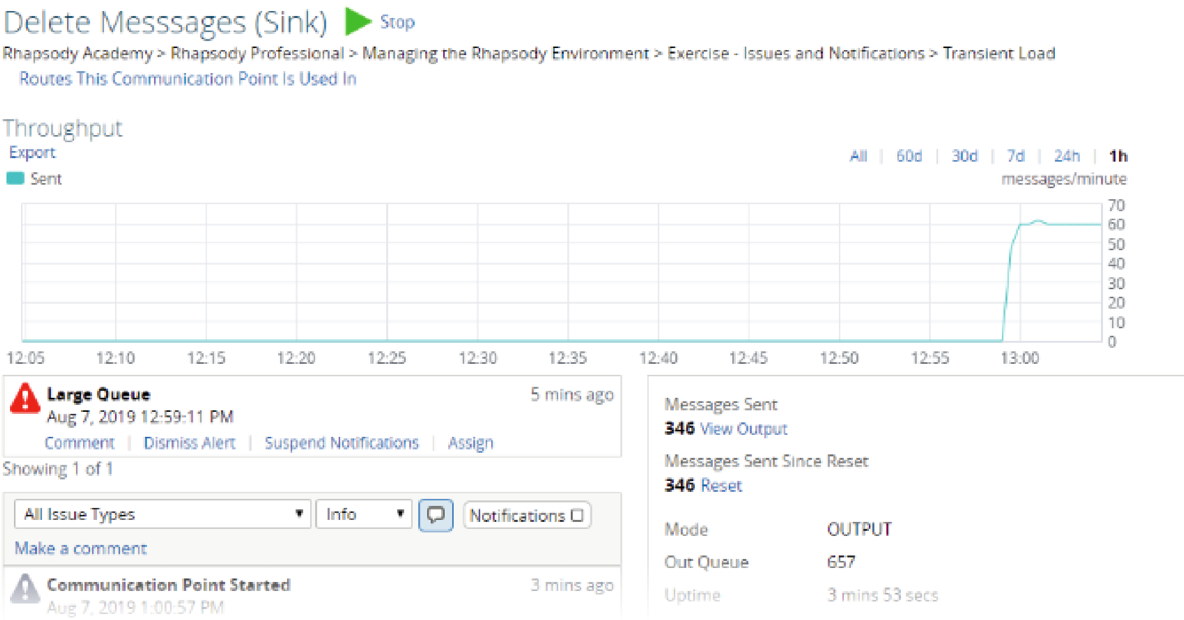
Open the **Patient Details DB Out** communication point details and then copy the complete ADTA04 data set (1000 messages) to the input directory
- C:\rhapsody\NotificationsEx in the default configuration.

This will cause two Alarms to be raised:

- Slow Message Processing
- Large Queue

Continue to monitor the component. As the message load drops, both Alarms will be resolved automatically by the system as messages are output and the load drops.

Once the load has dropped away and the state has returned to normal, copy the message files to the Input directory again; the Issues will be re-opened (rather than new Issues raised) as the issue has recurred within a short time of the previous event.

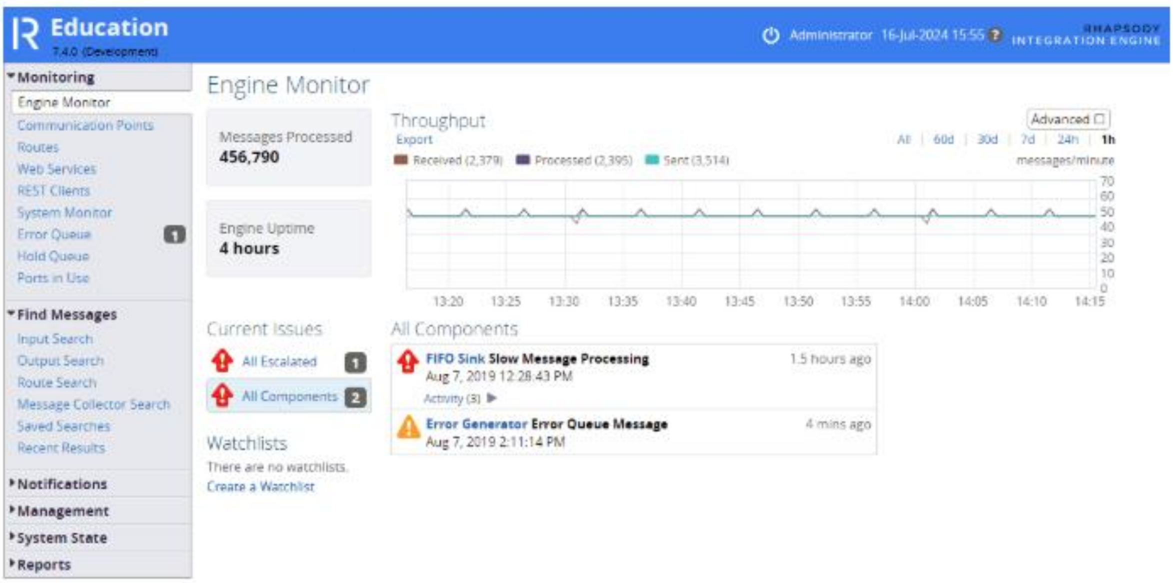


Large Queue notification on Delete Messages sink communication point

Threshold - Continuous

The **FIFO** route illustrates the Issue management during an event which is ongoing. The design of the route causes messages to be delayed during transmission through the route. Consequently, the long message latency causes an Issue to be raised.

Start all components of the FIFO route and open the **Engine Monitor** page of the Management Console. After a short interval, you should see the slow processing warning for the **FIFO Sink** open.



List of notifications on the Engine Monitor page

Select the link for the **FIFO Sink** to open the component details page.

This issue will remain open over time because the thresholds are set too low. Select **Dismiss** and note that the option is given to withhold Notifications for a period. You may choose to withhold if the Issue is occurring because the load is close to the threshold and is likely to breach it occasionally for a short time until it settles down again. This option will prevent spurious Notifications being issued when the issue is in hand.

Adjusting the Custom Thresholds

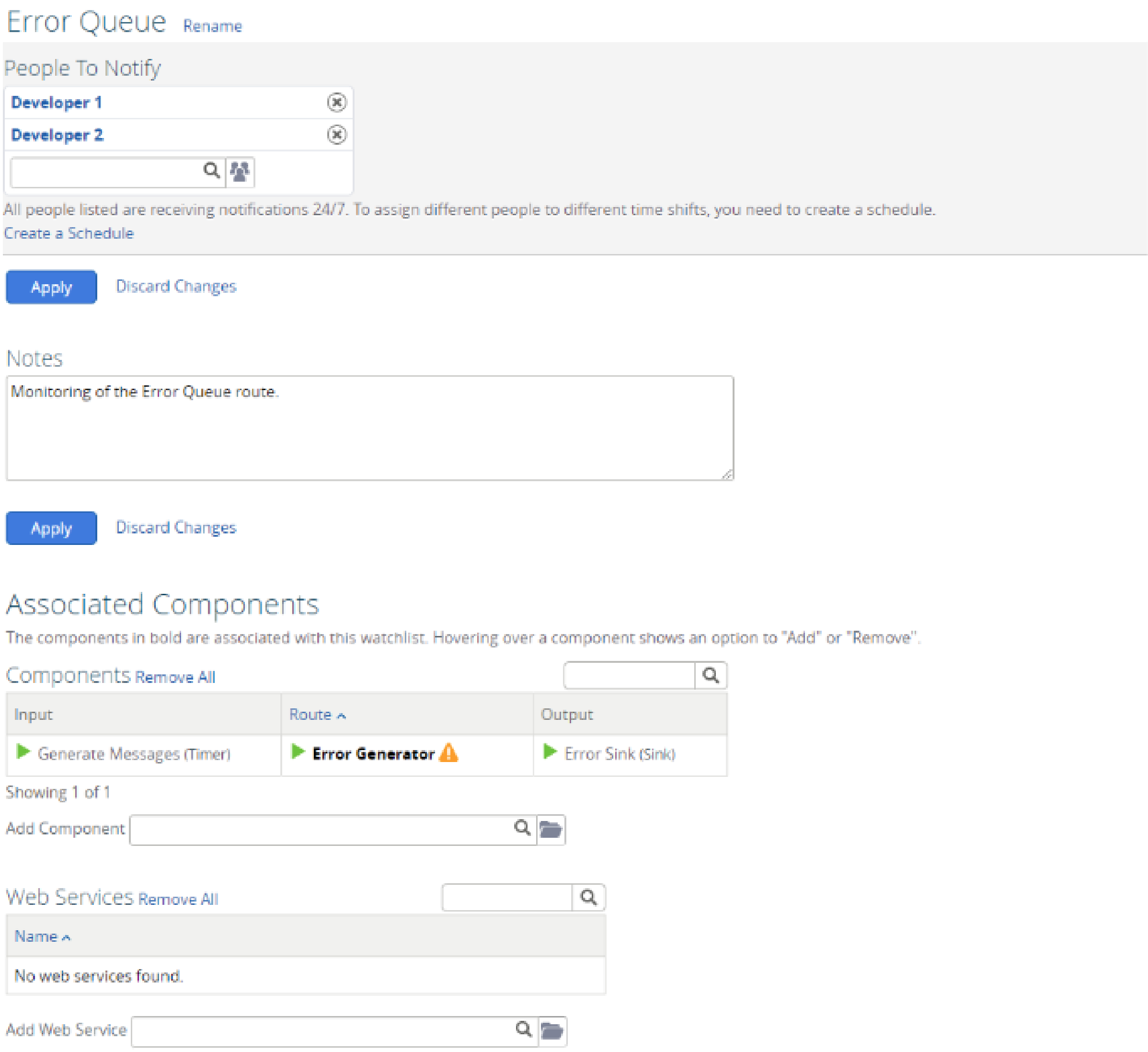
A notification has occurred because the threshold is set too low for the component. We will fix this by adjusting the thresholds for the component.

Open the **Custom Thresholds** link for the component and adjust the Warning and Alarm levels for the **Slow Message Processing** threshold to 50 (seconds) and 100 (seconds). Apply the changes and monitor the Issue. After a short interval, the Issue will automatically resolve.

Watchlists

Construct a Watchlist to monitor the **Error Generator** route, assigning both development users to the list. First, ensure the delivery details for both users have been set correctly.

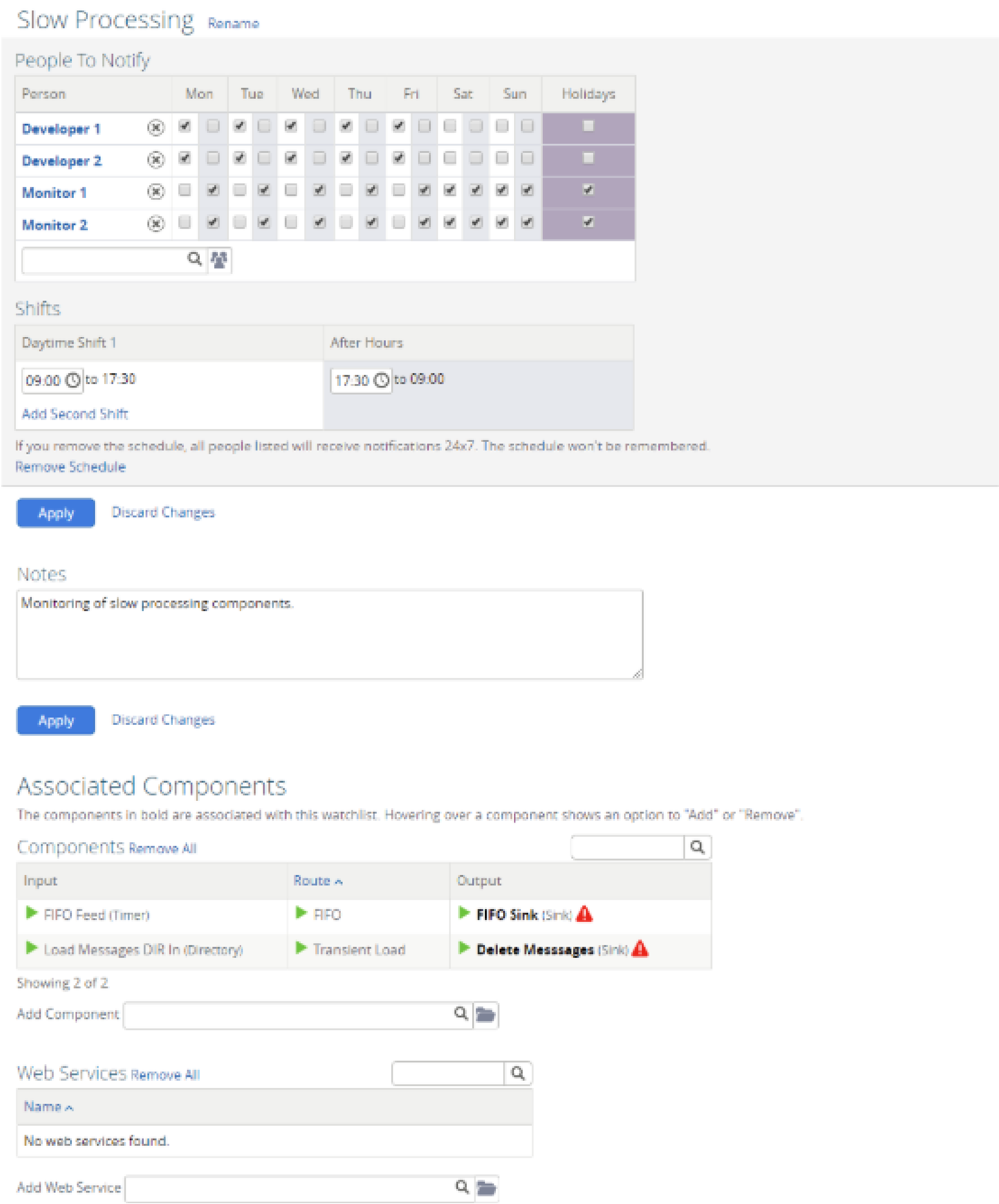
Start the Error Generator components and ensure that all the Notifications are received correctly.



Error Queue watchlist configuration

Create a Watchlist for the **Slow Processing** components. Create a schedule containing 2 shifts and assign the development users to the day shifts and the monitoring users to the overnight, weekend, and Holiday shifts.

Include the **FIFO Sink** and **Delete Messages** components in the list of components to be monitored.



Slow Processing watchlist configuration